

Advanced Algorithms for Optimization

LINFO2266

Linear Programming

Prof: Pierre Schaus
TAs: Alice Burlats, Amaury Guichard



Outline

- Lagrangian Relaxation: A quite generic technique to compute lower bounds
- Application to
 - Resource Constrained Shortest Path Problems (RCSPP)
 - The TSP (your favorite problem)

A linear programme is

maximize
subject to

$$\begin{aligned}x_1 + x_2 \\4x_1 - x_2 &\leq 8 \\2x_1 + x_2 &\leq 10 \\5x_1 - 2x_2 &\geq -2 \\x_1, x_2 &\geq 0\end{aligned}$$

1) The maximization of a **linear** function

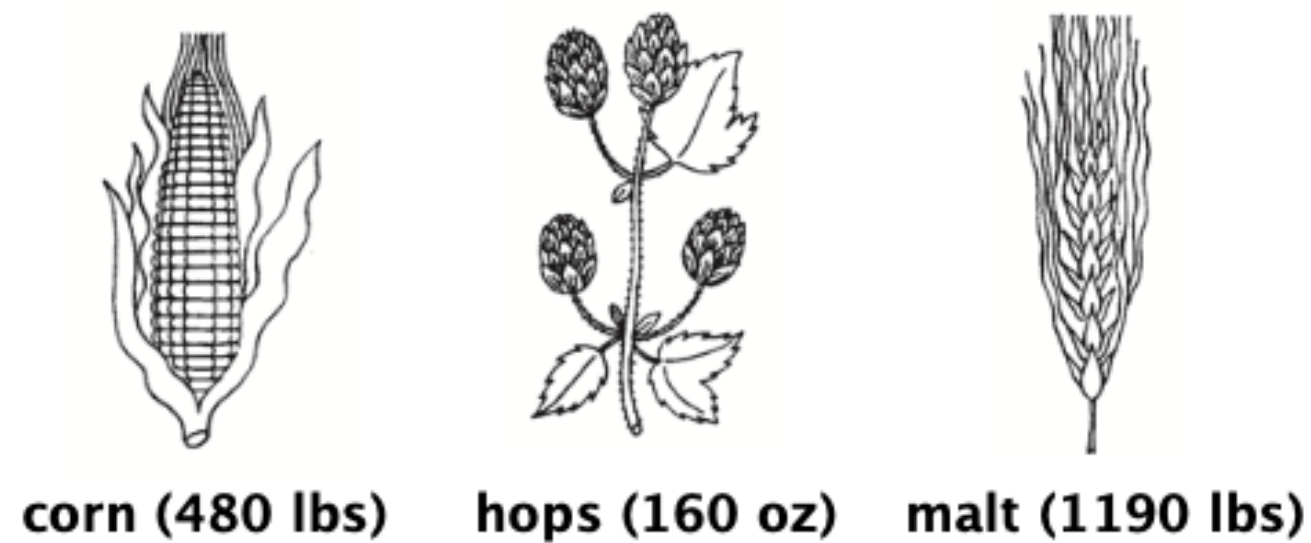
2) Over **linear** constraints

Matrix Notation

$$\begin{aligned}\text{maximize } & cx \\ \text{subject to } & Ax \leq b \\ & x \geq 0\end{aligned}$$

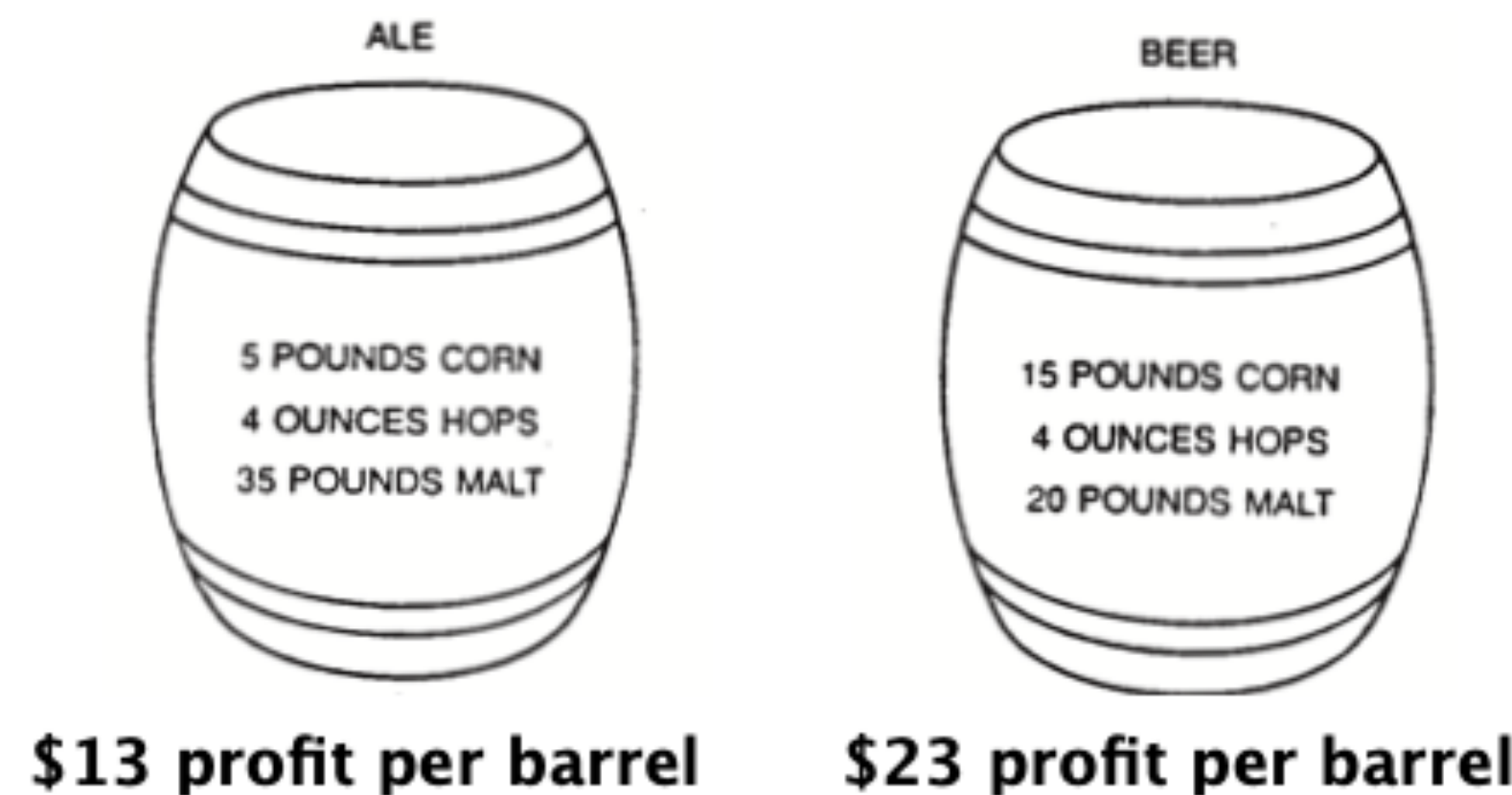
The Brewer Problem

- A small brewery produces **ale** and **beer** and wants to maximize profit
- Production limited by scarce resources (corn, hops, malt)



Capacity on each

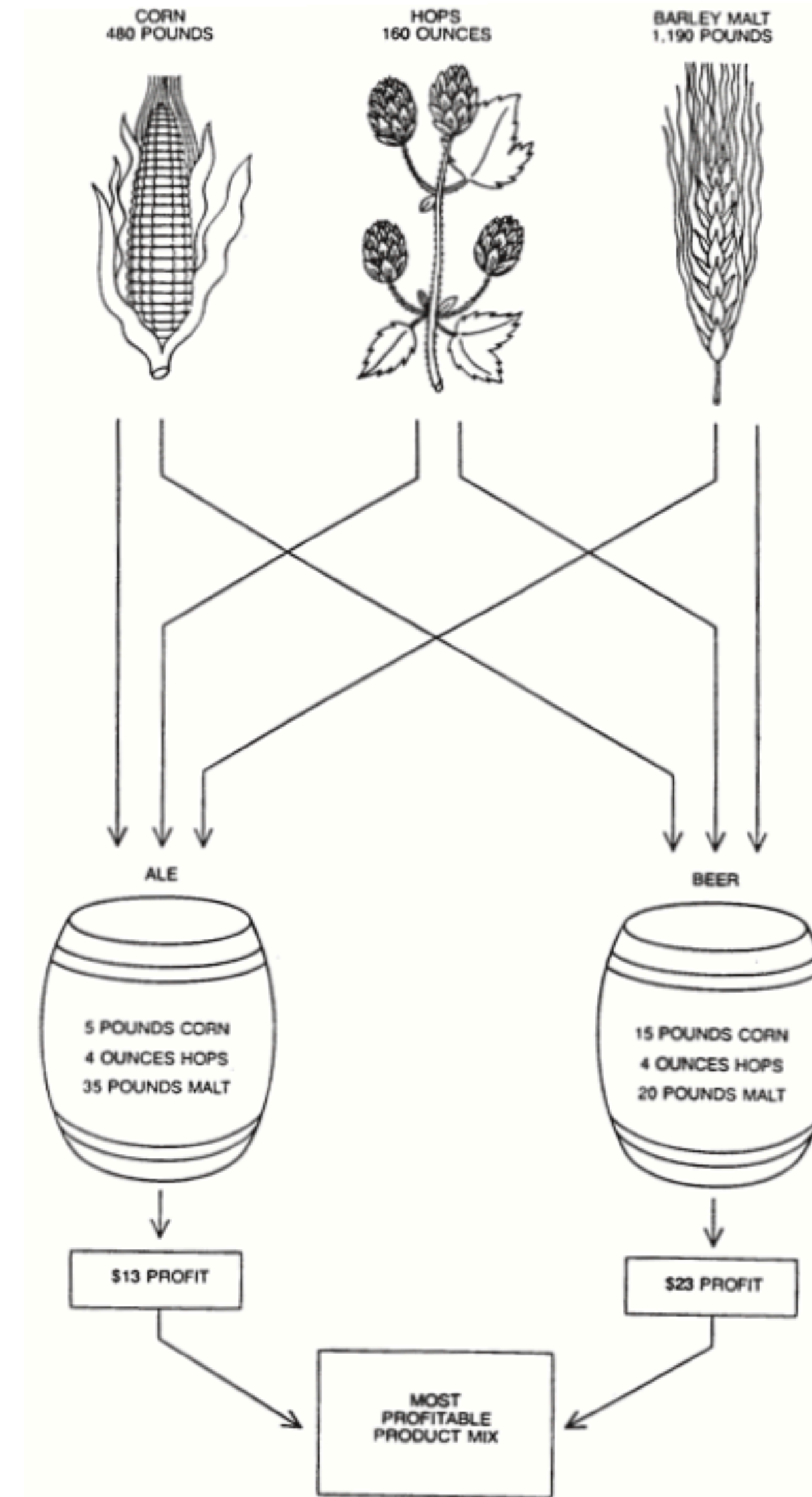
- The recipe for ale and beer requires different proportions of resources, and don't generate the same profit per barrel



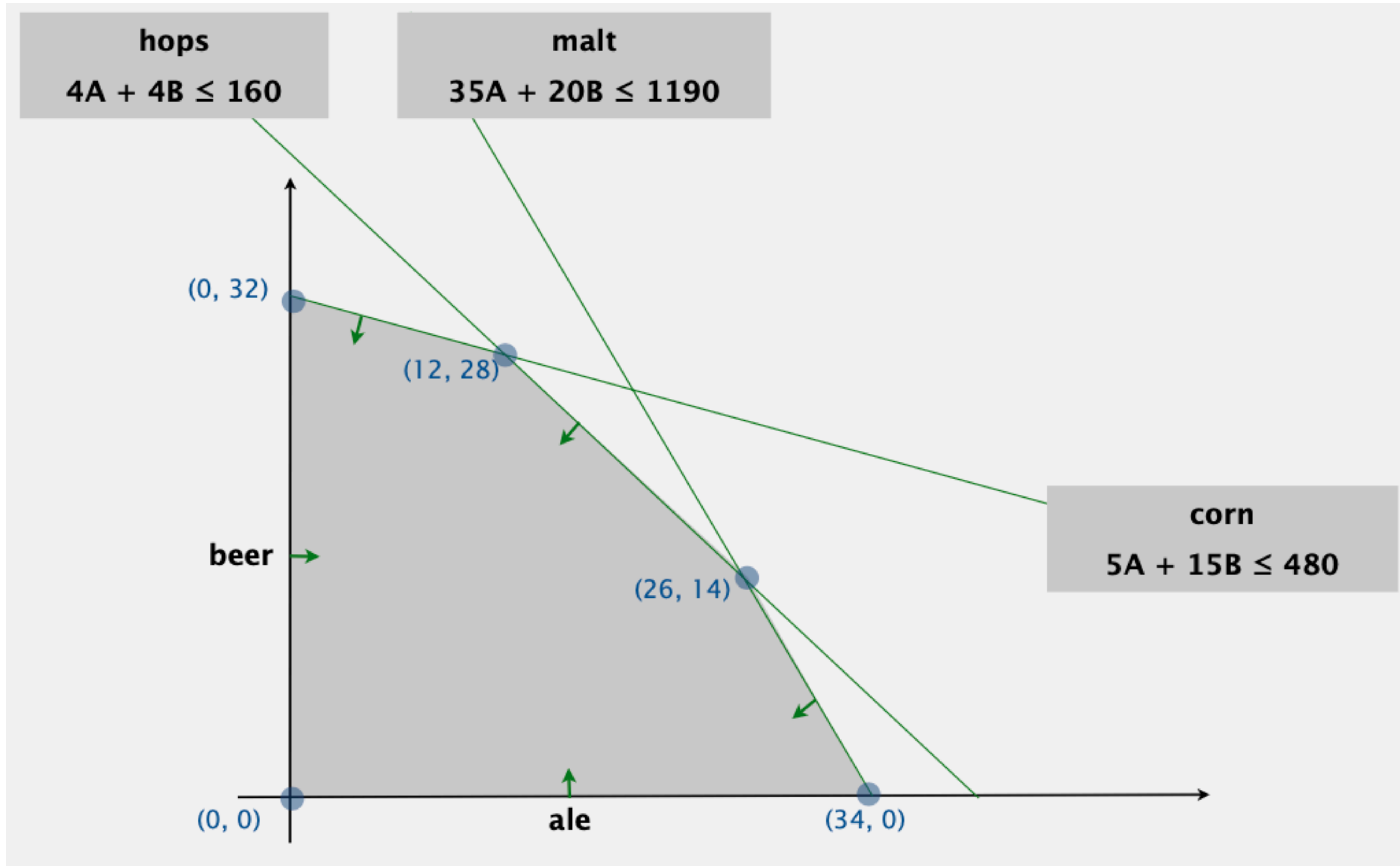
Brewer Problem: Linear Programming Formulation

- Variables: A = the number of barrels of ale, B = the number of barrels of beer.

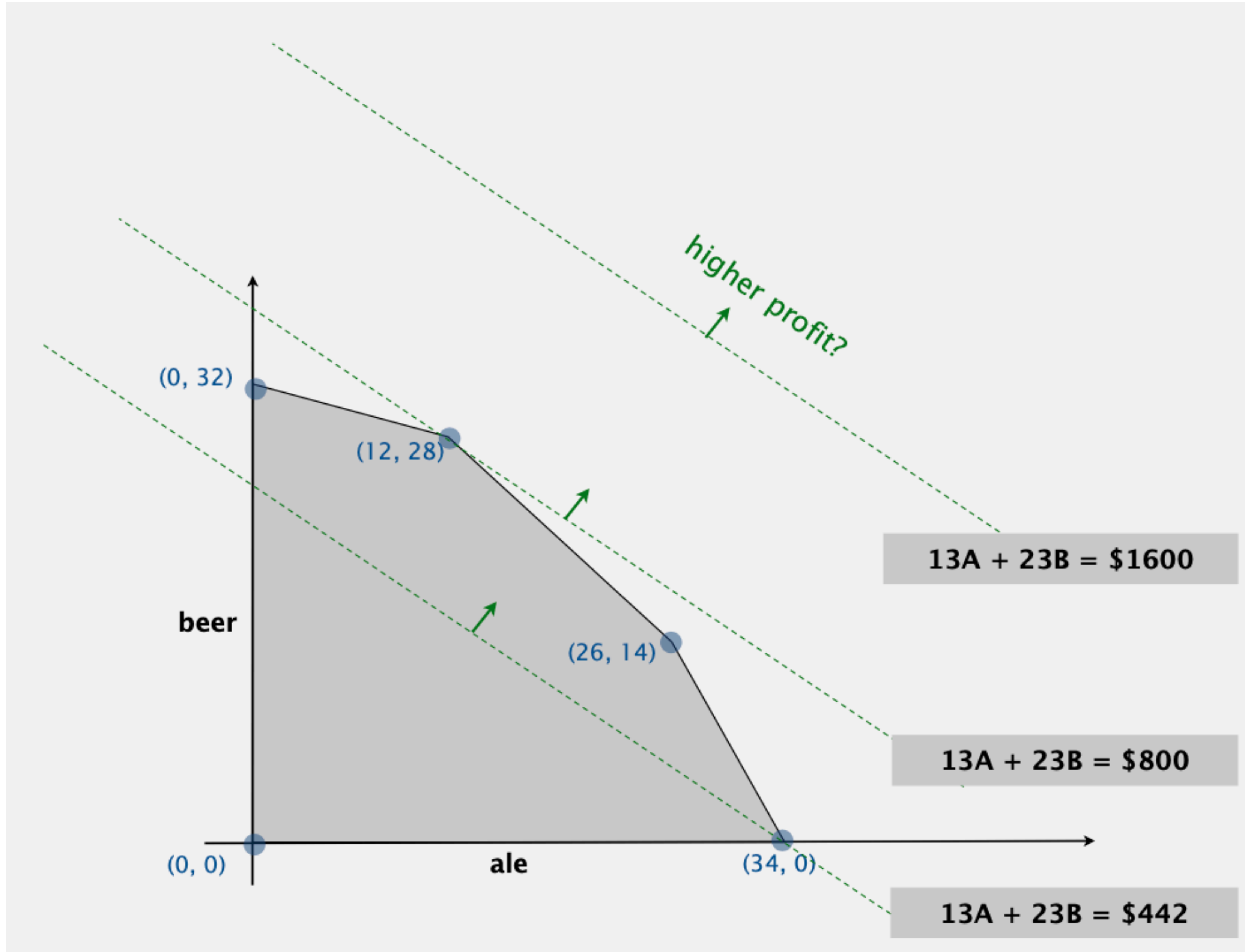
	ale		beer			
maximize	13A	+	23B			profits
subject to the constraints	5A	+	15B	≤	480	corn
	4A	+	4B	≤	160	hops
	35A	+	20B	≤	1190	malt
	A	,	B	≥	0	



Brewer Problem: Feasible Region



Brewer Problem: Objective Function



3D example, feasible region

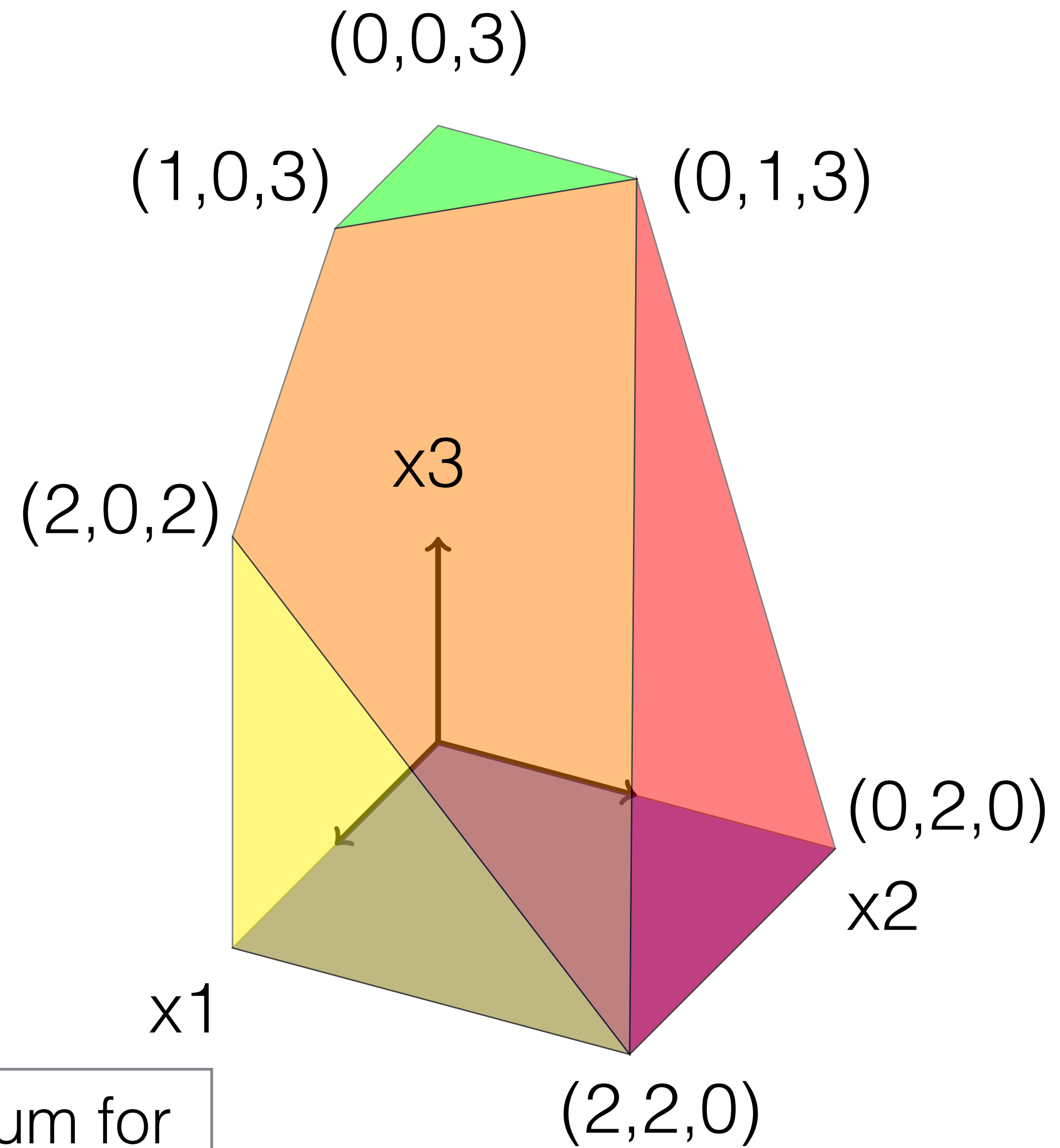
$$x_1 + x_2 + x_3 \leq 4$$

$$x_1 \leq 2$$

$$x_3 \leq 3$$

$$3x_2 + x_3 \leq 6$$

$$x_1, x_2, x_3 \geq 0$$



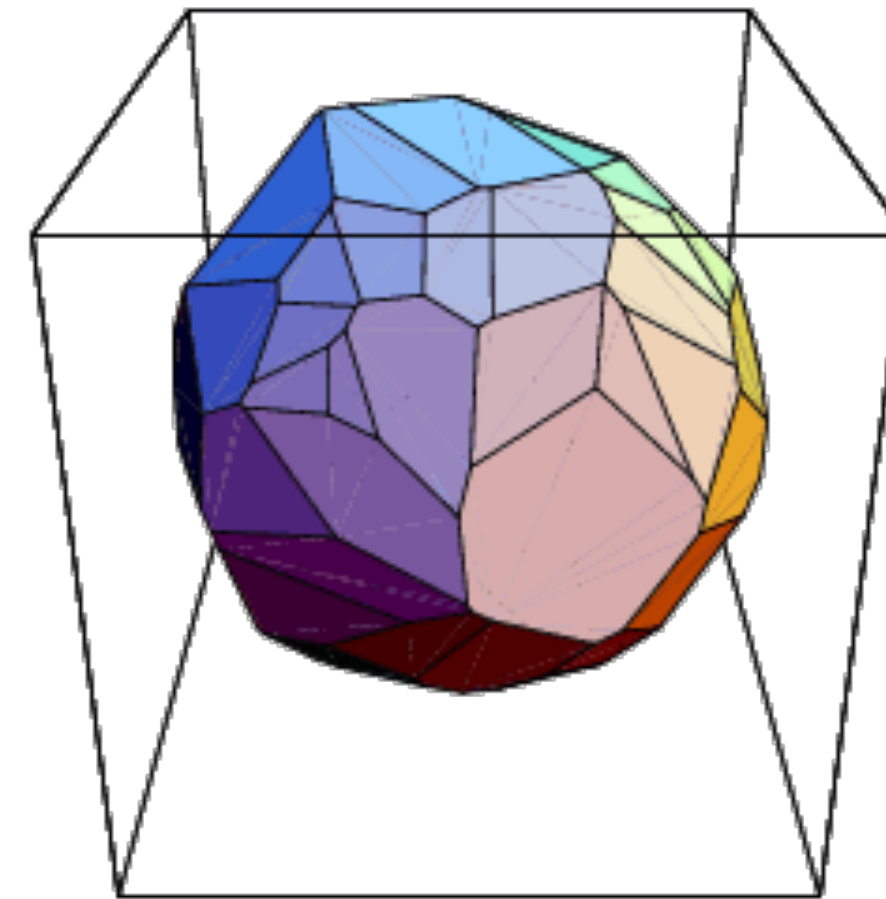
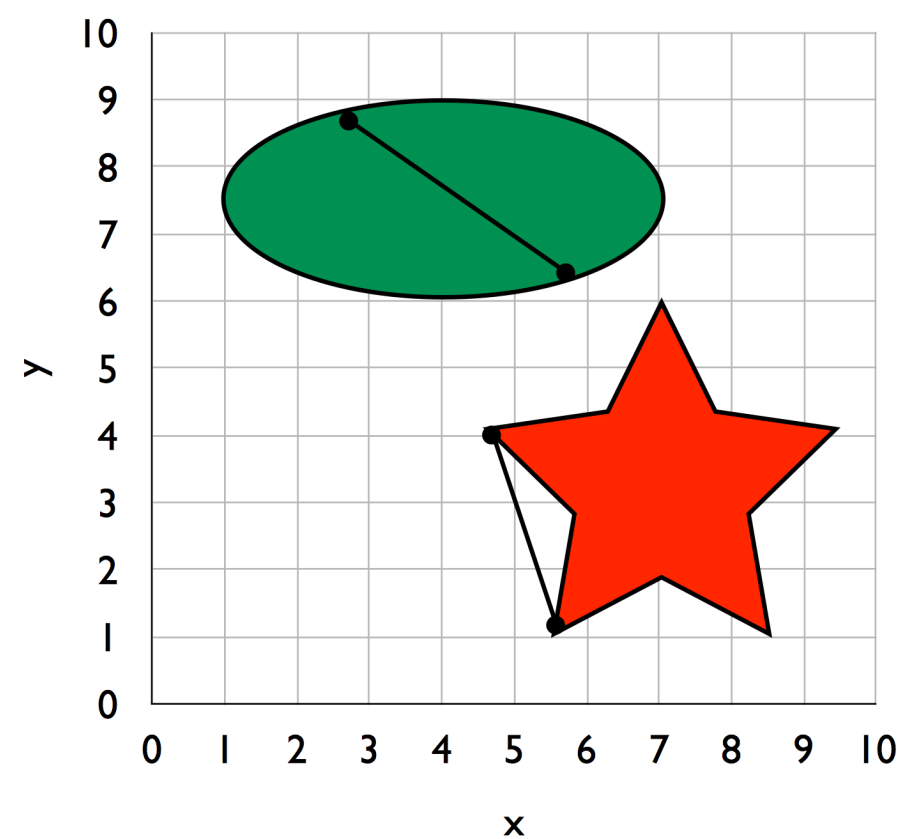
Exercise: What would be optimum for

- maximize $x_1 + x_2 + x_3$?
- maximize $x_1 + x_2$?

Convex Polytope

- The solution space of a linear system of linear equalities is a convex polytope.
- S is a set of points. S is convex iff for any point x and y in S , any convex combination is in S

$$(\alpha x + (1 - \alpha)y) \in S \text{ with } \alpha \in [0,1]$$



Theorem: Every point in a polytope is a convex combination of its vertices

Optimality is at vertices

maximize $c_1x_1 + \dots + c_nx_n$

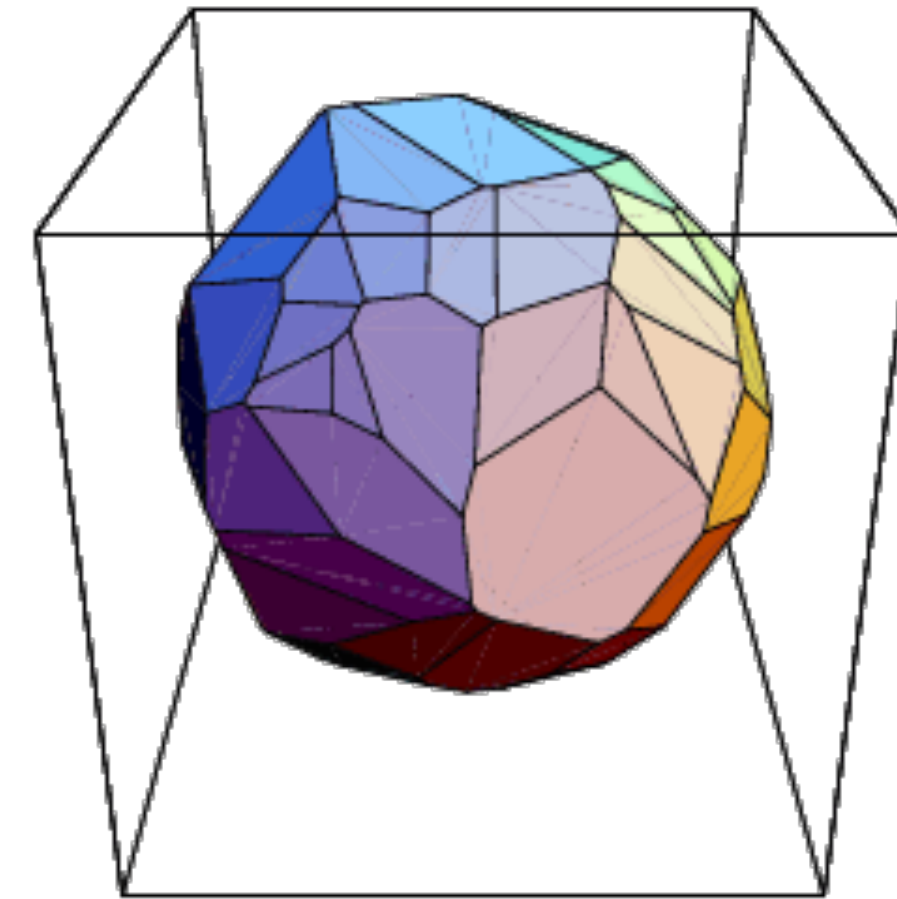
subject to

$$a_{11}x_1 + \dots + a_{1n}x_n \leq b_1$$

...

$$a_{m1}x_1 + \dots + a_{mn}x_n \leq b_m$$

$$x_i \geq 0 \quad (1 \leq i \leq n)$$



Theorem: At least one of the points where the objective value is maximal **is a vertex**.

Optimality at a vertex: Proof

Let x^* be the maximum. Since each point in a polytope is a convex combination of the vertices $v_1, \dots, v_t$ we have $x^* = \lambda_1 v_1 + \dots + \lambda_t v_t$

The objective value at optimality can be expressed as

$$cx^* = \lambda_1(cv_1) + \dots + \lambda_t(cv_t)$$

scalar product (c is a vector)

Assume that the maximum is not at a vertex, i.e.: $cx^* > cv_i \quad \forall i : 1 \leq i \leq t$, then

$$\begin{aligned} cx^* &= \lambda_1(cv_1) + \dots + \lambda_t(cv_t) \\ &< \lambda_1(cx^*) + \dots + \lambda_t(cx^*) \\ &= (\lambda_1 + \dots + \lambda_t)(cx^*) \\ &= cx^* \end{aligned}$$

each vertex is less good than x^*

contradiction

Hence, it must be the case that $x^* = v_i$ for some $1 \leq i \leq t$.

A first Algorithm

- Enumerate all the vertices
- Select the one with the largest objective value
- What do you think about this ? 🤔

Number of of vertices

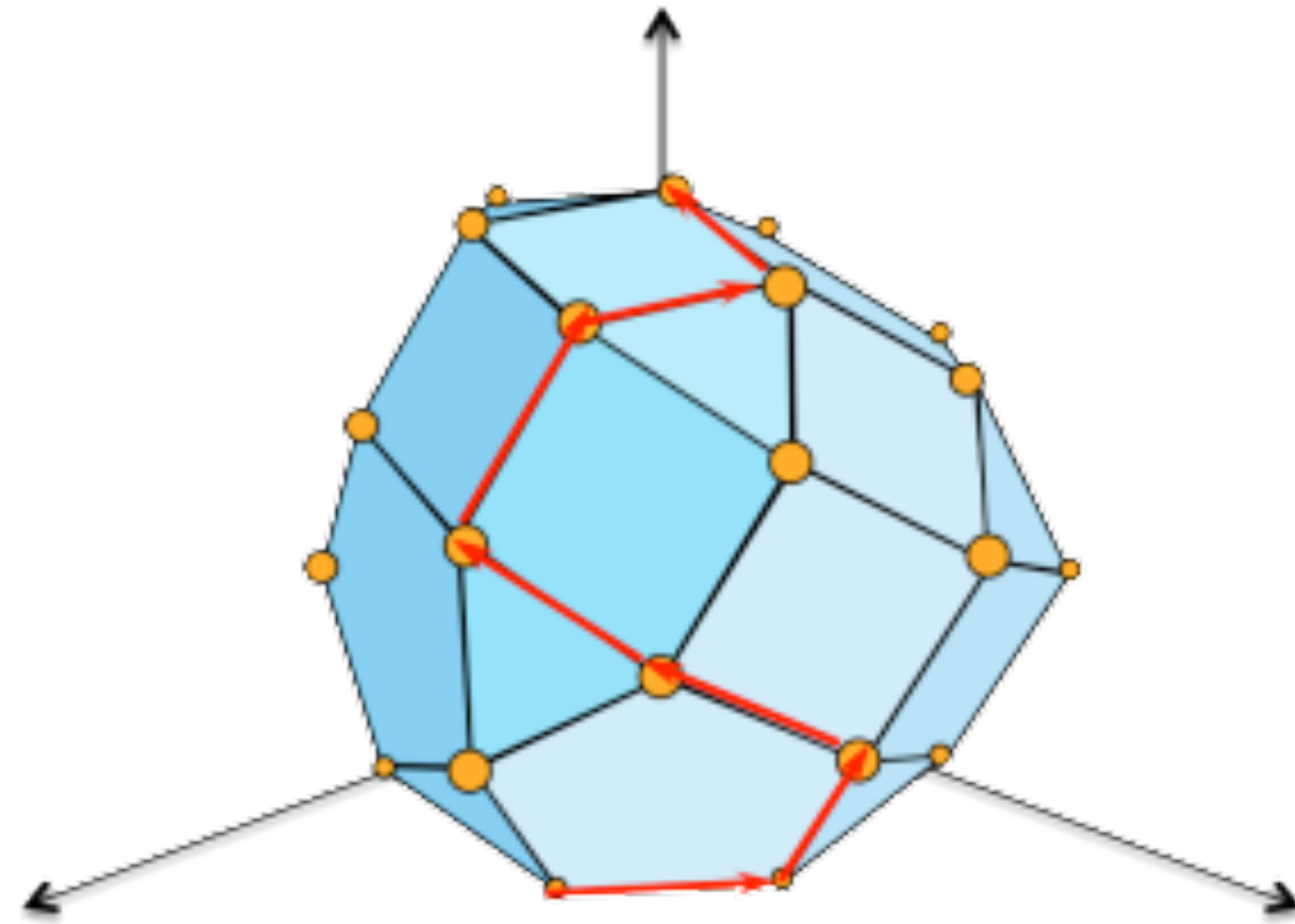
- In 1D, the unit interval $[0, 1]$ has 2 endpoints. Intersection of 2 half-spaces.
- In 2D, the unit square has 4 vertices. Intersection of 4 half-spaces
- In 3D, the unit cube has 8 vertices, defined by the intersections of the 6 half-spaces ...
- In n D, the unit hypercube has 2^n vertices defined by the intersection of $2n$ half spaces.

The pattern is clear: The number of vertices can grow exponentially with the number of inequalities.

Conclusion: enumerating all the vertices is impractical 💣.

Solution: A clever algorithm, called Simplex 🥳

Simplex Algorithm



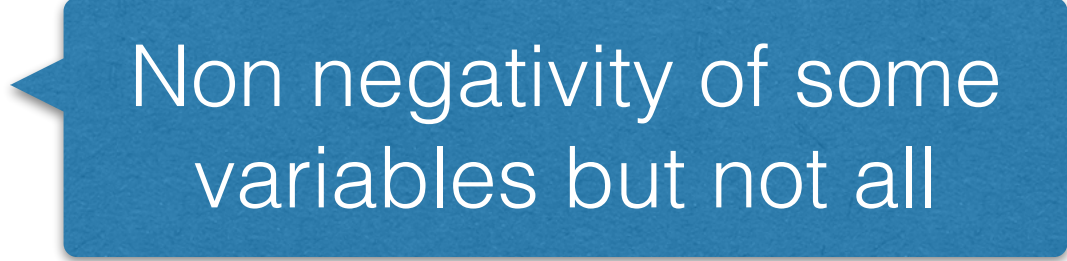
- Move from one vertex, to a neighboring vertex with an improving objective function.
- Move until no more improving neighbor vertex
- An optimal vertex is always reached because of convexity of polytope

The different forms of Linear Program

maximize $2x_1 - 3x_2$

subject to $x_1 + x_2 = 7$  equality constraint

$x_1 - 2x_2 \leq 4$  inequality constraint

$x_1 \geq 0$  Non negativity of some variables but not all

- This is a bit messy, let's make a unique form, called ***canonical form***

Canonical Form (only $\leq$ constraints and, all vars ≥ 0)

$$\text{maximize } \sum_{j=1}^n c_j x_j$$

n variables, m constraints

$$\text{subject to } \sum_{j=1}^n a_{ij} x_j \leq b_i \text{ for } i = 1, 2, \dots, m$$

$$x_j \geq 0 \text{ for } j = 1, 2, \dots, n$$

non negativity constraints for all variables

💡 Always possible to convert a linear program into canonical form

Converting into canonical form is an easy process

- If a variable x_j has no non-negativity constraint, replace each of its occurrence by

$$x'_j - x''_j$$

- Convert equality constraints (=) into two ($\geq$, $\leq$)
- Exercise: Convert this into standard form

$$\begin{array}{ll}\text{maximize} & 2x_1 - 3x_2 \\ \text{subject to} & x_1 + x_2 = 7 \\ & x_1 - 2x_2 \leq 4 \\ & x_1 \geq 0\end{array}$$

Not yet the panacea

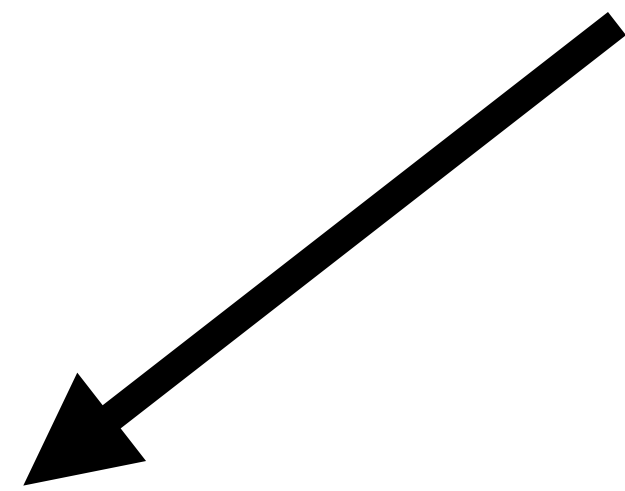
- For the solving, it is easier to deal with equality constraint.
- After all, we know how to solve a system of linear equations (Gauss-Jordan)
- Let's create another form, called the ***standard form*** with
 - equality constraints only, and
 - non negativity constraints on all the variables

Standard form (only non-negative constraints are inequalities)

$$\text{maximize } \sum_{j=1}^n c_j x_j$$

$$\text{subject to } \sum_{j=1}^n a_{ij} x_j \leq b_i \text{ for } i = 1, 2, \dots, m$$

$$x_j \geq 0 \text{ for } j = 1, 2, \dots, n$$



$$x_{n+i} = b_i - \sum_{j=1}^n a_{ij} x_j$$

$$x_{n+i} \geq 0$$

Introduce one **slack variable**/inequality
If slack = 0, we say the constraint is tight

n+m variables, m constraints

From Canonical Form to Standard Form

Canonical form

$$\begin{array}{ll}\text{maximize} & 2x_1 - 3x_2 \\ \text{subject to} & x_1 + x_2 = 7 \\ & x_1 - 2x_2 \leq 4 \\ & x_1 \geq 0\end{array}$$

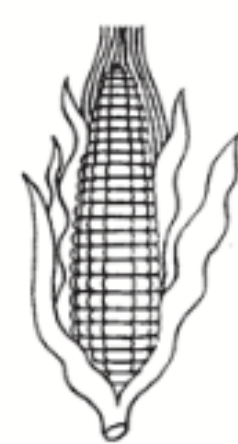
$$\begin{array}{ll}\text{maximize} & 2x_1 - 3x_2 + 3x_3 \\ \text{subject to} & x_1 + x_2 - x_3 \leq 7 \\ & -x_1 - x_2 + x_3 \leq -7 \\ & x_1 - 2x_2 + 2x_3 \leq 4 \\ & x_1, x_2, x_3 \geq 0\end{array}$$

Standard form

$$\begin{array}{ll}\text{maximize} & 2x_1 - 3x_2 + 3x_3 \\ \text{subject to} & x_4 = 7 - x_1 - x_2 + x_3 \\ & x_5 = -7 + x_1 + x_2 - x_3 \\ & x_6 = 4 - x_1 + 2x_2 - 2x_3 \\ & x_1, x_2, x_3, x_4, x_5, x_6 \geq 0\end{array}$$

Let's come back to the Brewer Problem*

- A small brewery produces ale and beer and wants to maximize profit
- Production limited by scarce resources (corn, hops, malt)



corn (480 lbs)



hops (160 oz)



malt (1190 lbs)

- The recipe for ale and beer require different proportions of resources



\$13 profit per barrel



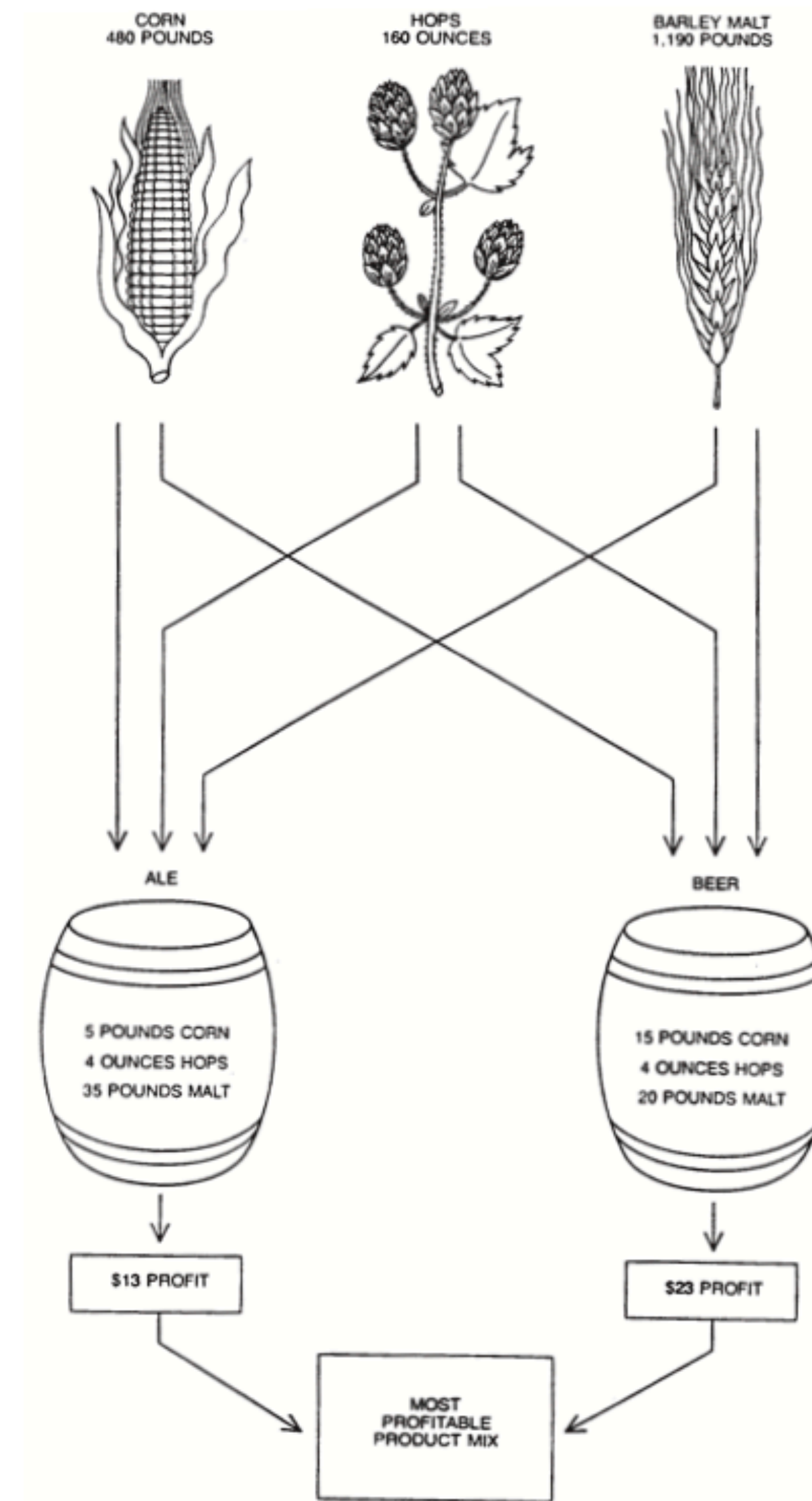
\$23 profit per barrel

*Slides from Sedgewick and Wayne, Algorithms part 2, Coursera

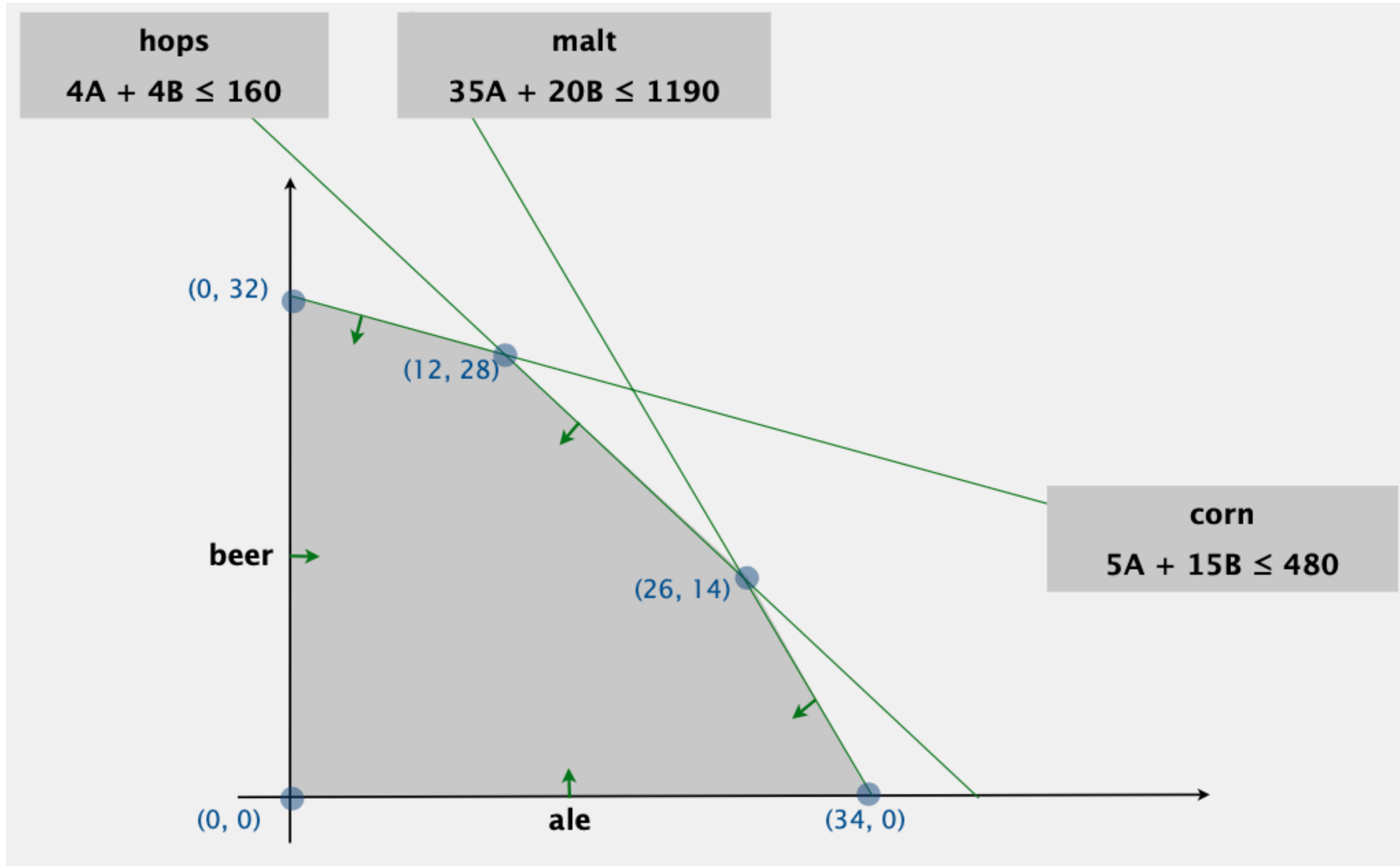
Brewer Problem: Linear Programming Formulation

- Variables: A = the number of barrels of ale, B = the number of barrels of beer.

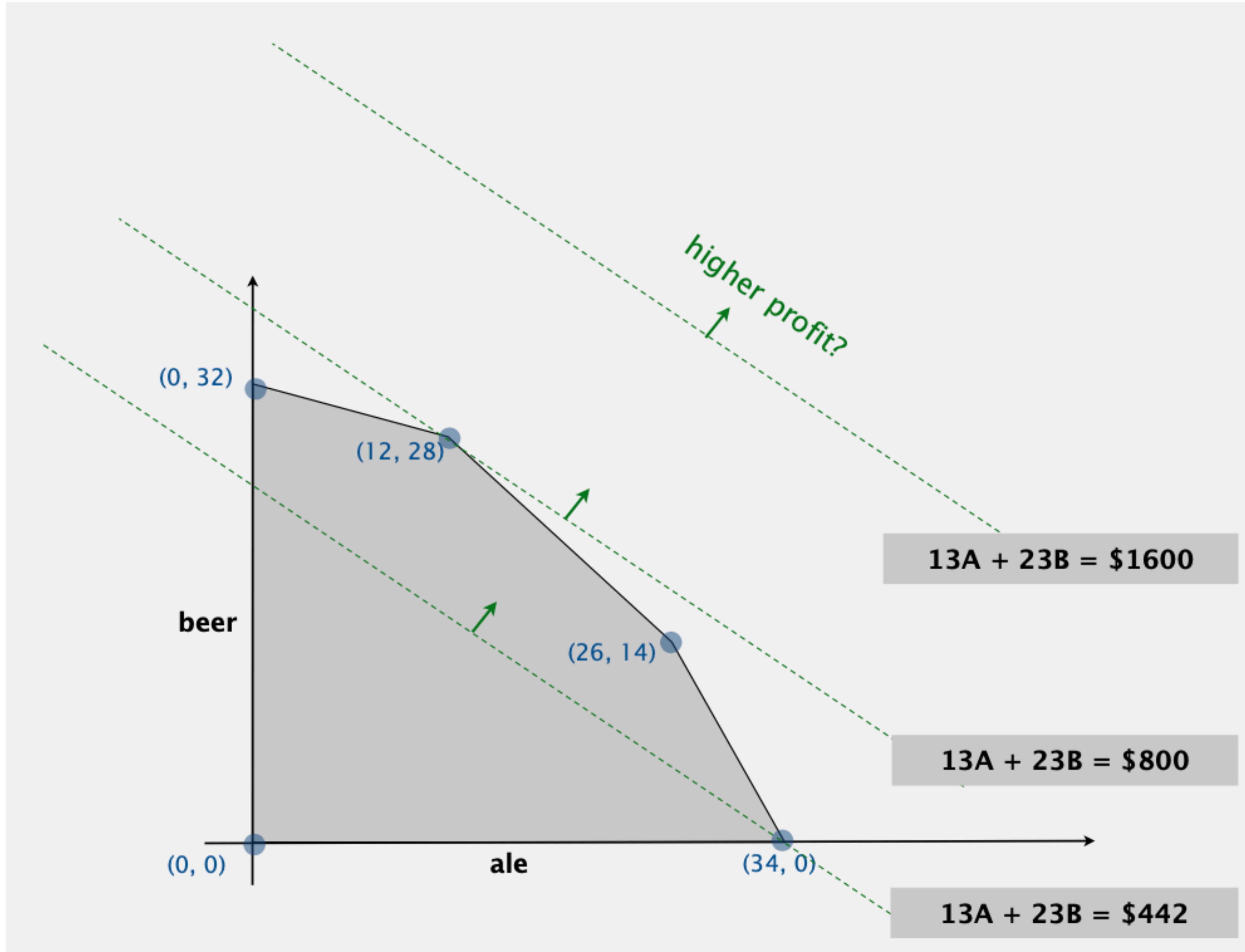
	ale		beer			
maximize	13A	+	23B			profits
subject to the constraints	5A	+	15B	≤	480	corn
	4A	+	4B	≤	160	hops
	35A	+	20B	≤	1190	malt
	A	,	B	≥	0	



Brewer Problem: Feasible Region



Brewer Problem: Objective Function



Brewer Problem: Standard Form

- Add three slack variables, S_C , S_H , S_M for the crop, hop and malt constraints
- Introduce variable Z for the objective function

Canonical Form

maximize $13A + 23B$

subject to the constraints

$$\begin{aligned} 5A + 15B &\leq 480 \\ 4A + 4B &\leq 160 \\ 35A + 20B &\leq 1190 \\ A, B &\geq 0 \end{aligned}$$

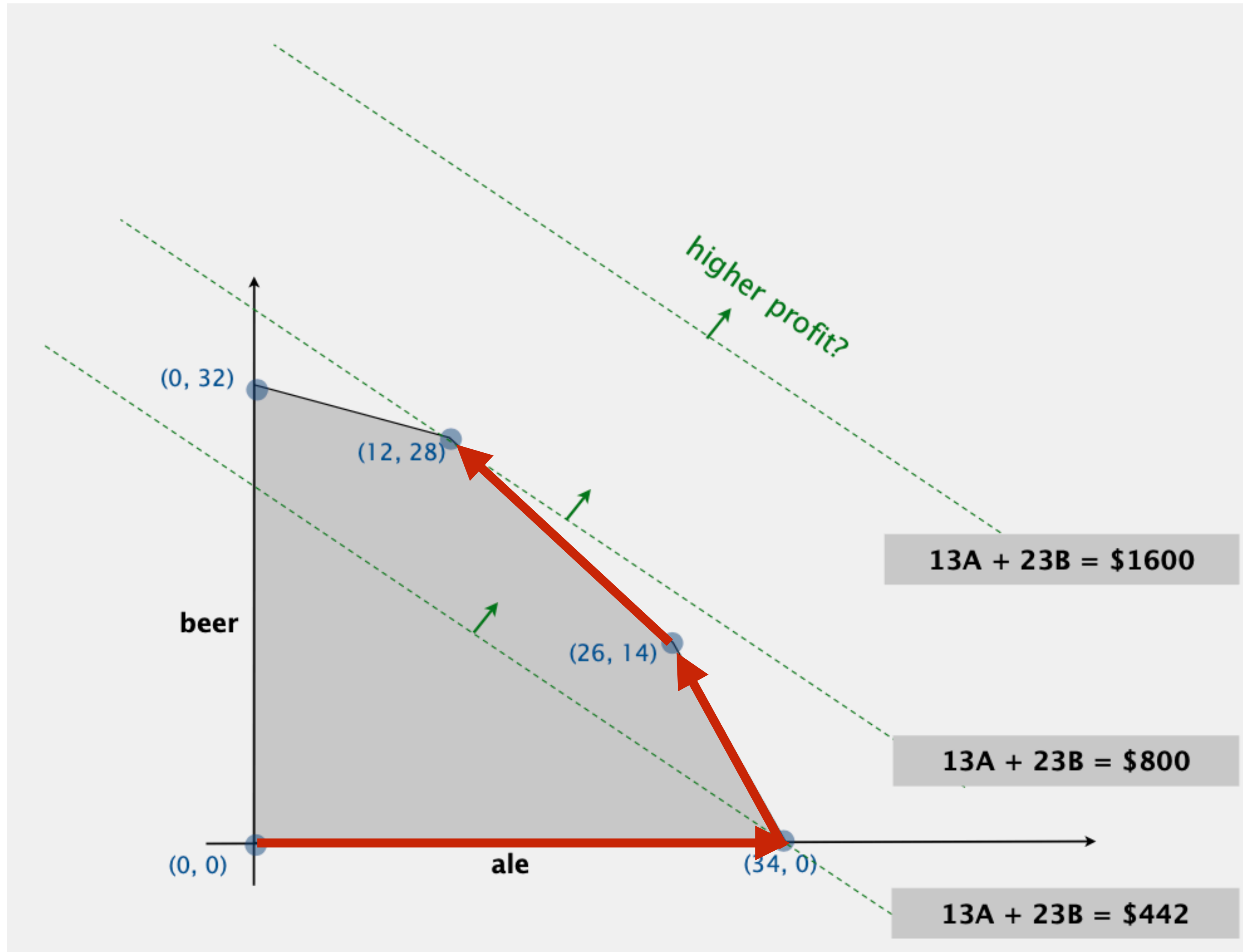
Standard Form

maximize Z

subject to the constraints

$$\begin{aligned} 13A + 23B & - Z = 0 \\ 5A + 15B + S_C & = 480 \\ 4A + 4B + S_H & = 160 \\ 35A + 20B + S_M & = 1190 \\ A, B, S_C, S_H, S_M & \geq 0 \end{aligned}$$

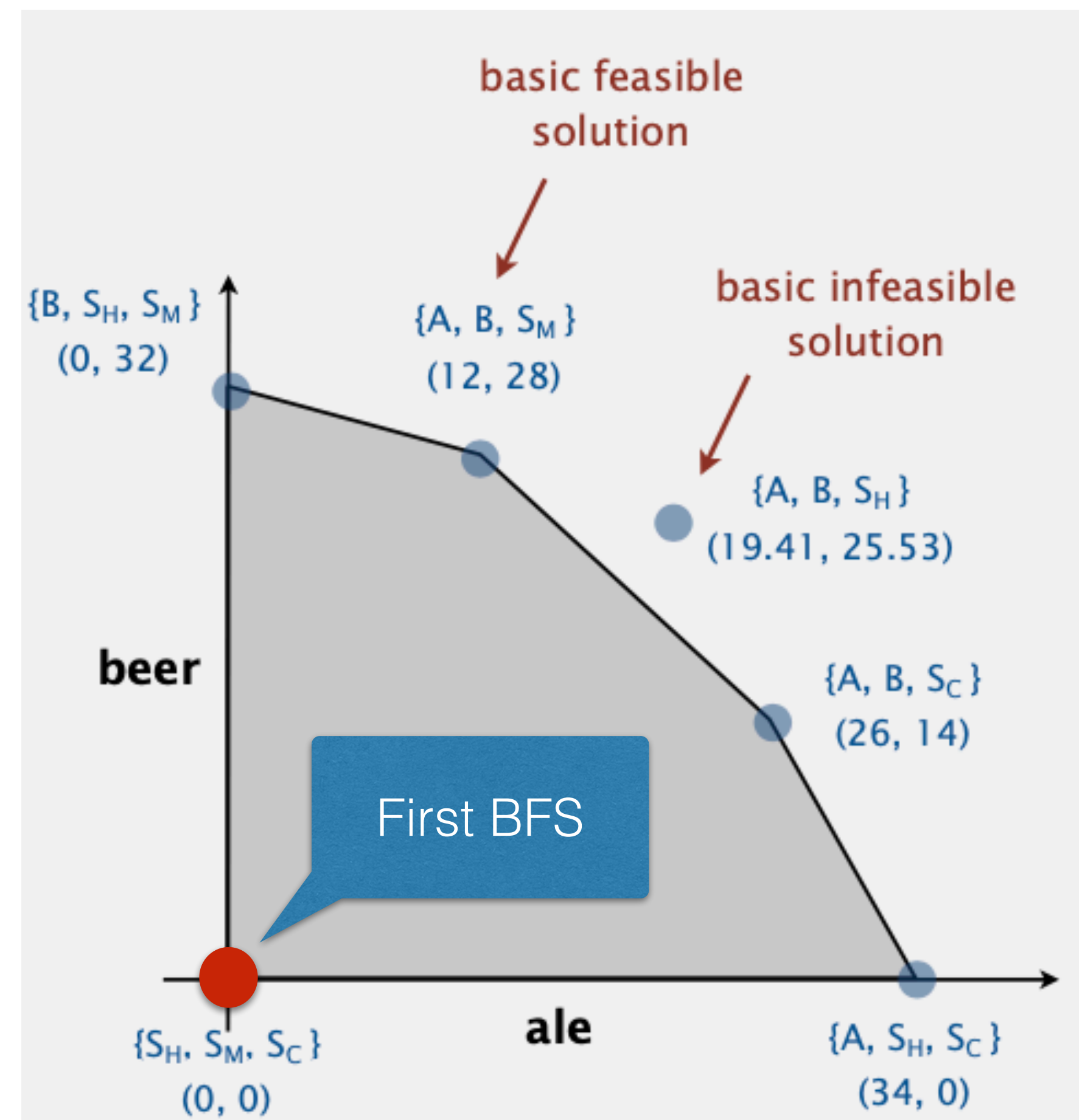
Vertices explored by the simplex algorithm



First Basic Feasible Solution (BFS)

- A subset $m=3$ of the $n=6$ variables
- Only those (basic) variables are non zero
- It corresponds to a feasible solution

maximize	Z										
	$13A$	$+$	$23B$					$-$	Z	$=$	0
subject to the constraints	$5A$	$+$	$15B$	$+$	S_C					$=$	480
	$4A$	$+$	$4B$			$+$	S_H			$=$	160
	$35A$	$+$	$20B$					$+$	S_M	$=$	1190
	A	,	B	,	S_C	,	S_H	,	S_M	$\geq$	0



- First basis, start with slack variables $\{S_C, S_H, S_M\}$ as the basis
- A and $B = 0$ $\{S_C=480, S_H=160, S_M=119\}$ can be read from the tableau, no algebra needed

Pivot 1

- Take one non basic variable and turn it into a basic variable to improve the solution
- Let us choose B to enter into the basis. If B increases, Z must increase.
- What variable does B replace ? Answer: S_C because if B increases, S_C , S_H and S_M must decrease but S_C will hit zero first.
- Min ratio rule ($S_C:480/15=32$, $S_H:160/4=40$, $S_M:1190/20=59.5$)

maximize	Z									
	13A	+	23B					-	Z	= 0
subject	5A	+	15B	+	S_C					= 480
to the	4A	+	4B			+	S_H			= 160
constraints	35A	+	20B				+	S_M	= 1190	
	A	,	B	,	S_C	,	S_H	,	S_M	≥ 0

pivot

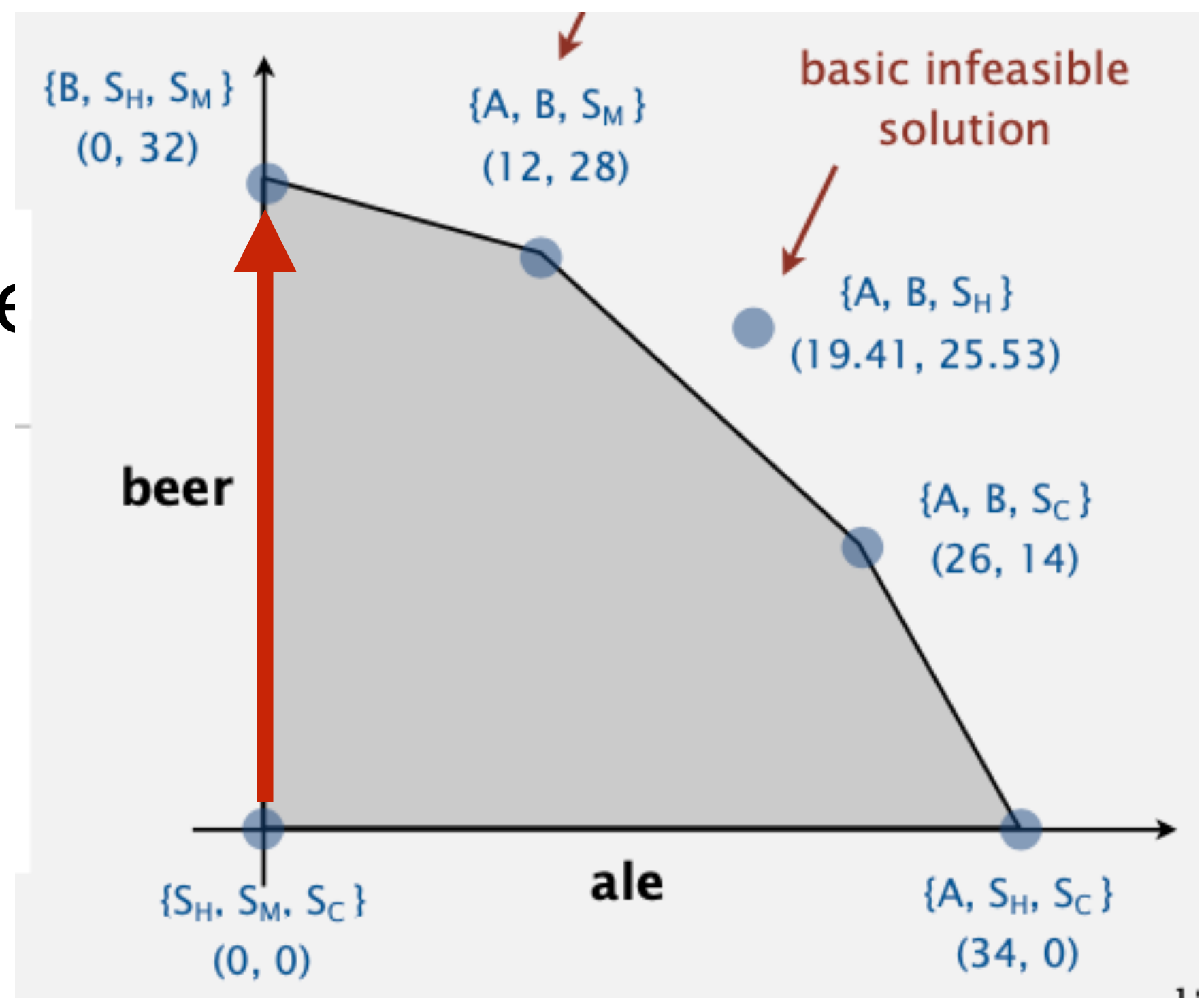
Pivot 1 (cont)

maximize	Z															
	13A	+	23B								-	Z	=	0		
subject	5A	+	15B	+	S _C								=	480		
to the	4A	+	4B			+	S _H						=	160		
constraints	35A	+	20B					+	S _M				=	1190		
	A	,	B	,	S _C	,	S _H	,	S _M				≥	0		

pivot

Substitute $B = (1/15) (480 - 5A - S_C)$ and add B into the eliminate B in 1st, 3rd, and 4th equations)

maximize	Z															
	(16/3) A			-	(23/15) S _C						-	Z	=	-736		
subject	(1/3) A	+	B	+	(1/15) S _C								=	32		
to the	(8/3) A			-	(4/15) S _C	+	S _H						=	32		
constraints	(85/3) A			-	(4/3) S _C			+	S _M				=	550		
	A	,	B	,	S _C	,	S _H	,	S _M				≥	0		

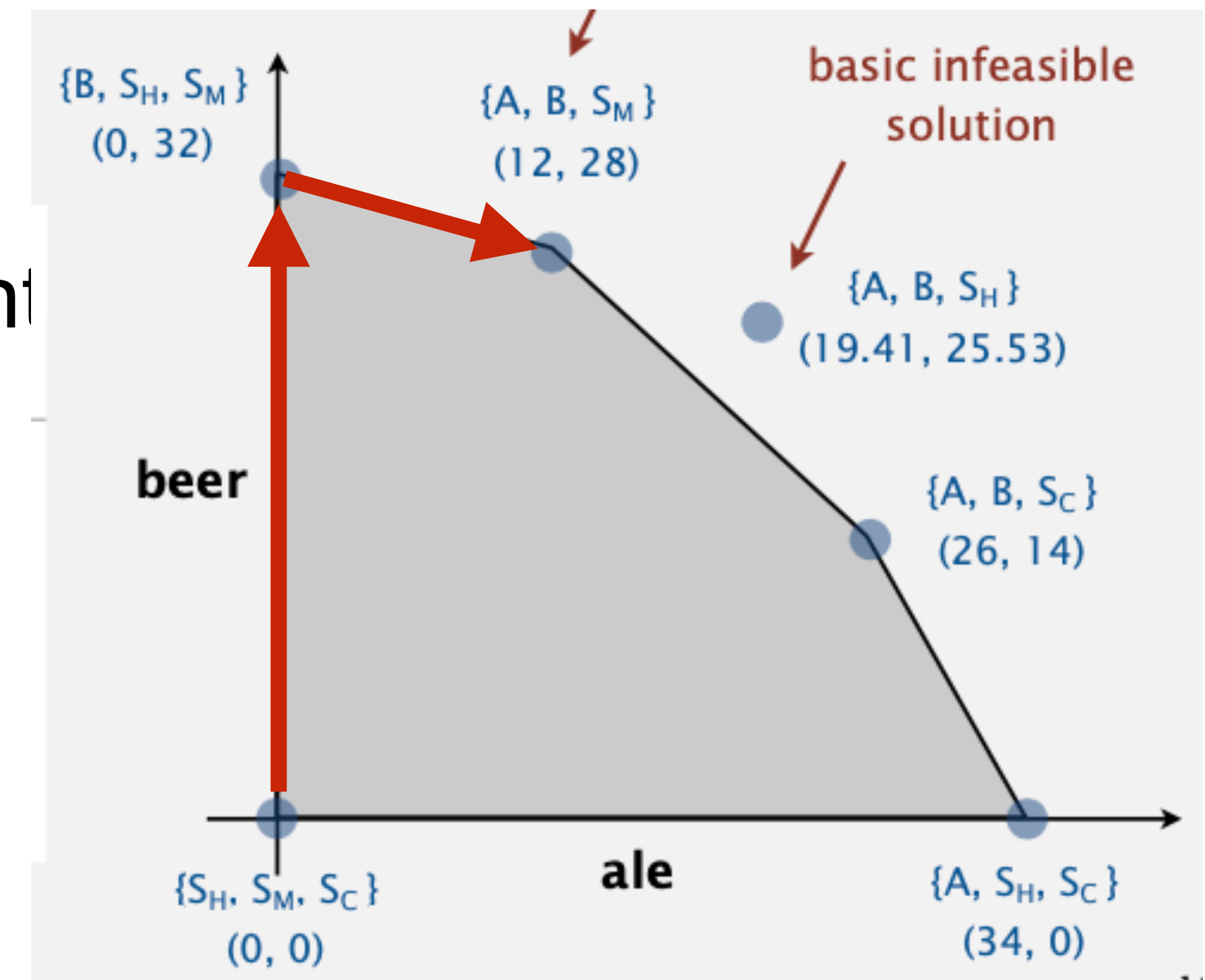


Pivot 2

maximize	Z								
	(16/3) A		-	(23/15) S _C			-	Z	= -736
subject	(1/3) A	+	B	+	(1/15) S _C			=	32
to the	(8/3) A			-	(4/15) S _C	+	S _H	=	32
constraints	(85/3) A			-	(4/3) S _C		+	S _M	= 550
	A	,	B	,	S _C	,	S _H	,	S _M ≥ 0

Substitute $A = (3/8) (32 + (4/15) S_C - S_H)$ and add A into 3rd equation, eliminate A in 1st, 2nd, and 4th equations)

[illegible]



Optimality

- Basis = $\{A, B, S_M\}$, $Z = 800$, $A = 12$, $B = 28$
- Optimal solution. Stop pivoting when no objective coefficient is positive. Why ?

maximize	Z								
			$-$	S_C	$-$	$2 S_H$		$- Z$	$= -800$
subject		B	$+$	$(1/10) S_C$	$+$	$(1/8) S_H$		$=$	28
to the		A	$-$	$(1/10) S_C$	$+$	$(3/8) S_H$		$=$	12
constraints			$-$	$(25/6) S_C$	$-$	$(85/8) S_H + S_M$		$=$	110
		A	$,$	B	$,$	S_C	$,$	S_H	$, S_M \geq 0$

- Any feasible solution satisfies the system of equations.
- In particular: $Z = 800 - S_C - 2S_H$. Since $S_C, S_H \geq 0$, $Z^* \leq 800$
- Current BFS has $Z=800$, thus it is optimal

Let's translate this into Java (using 2D arrays)

maximize

Z

13A + 23B

subject to the constraints

5A + 15B + S_C

4A + 4B + S_H

35A + 20B + S_M

A , B , S_C , S_H , S_M

- Z = 0

= 480

= 160

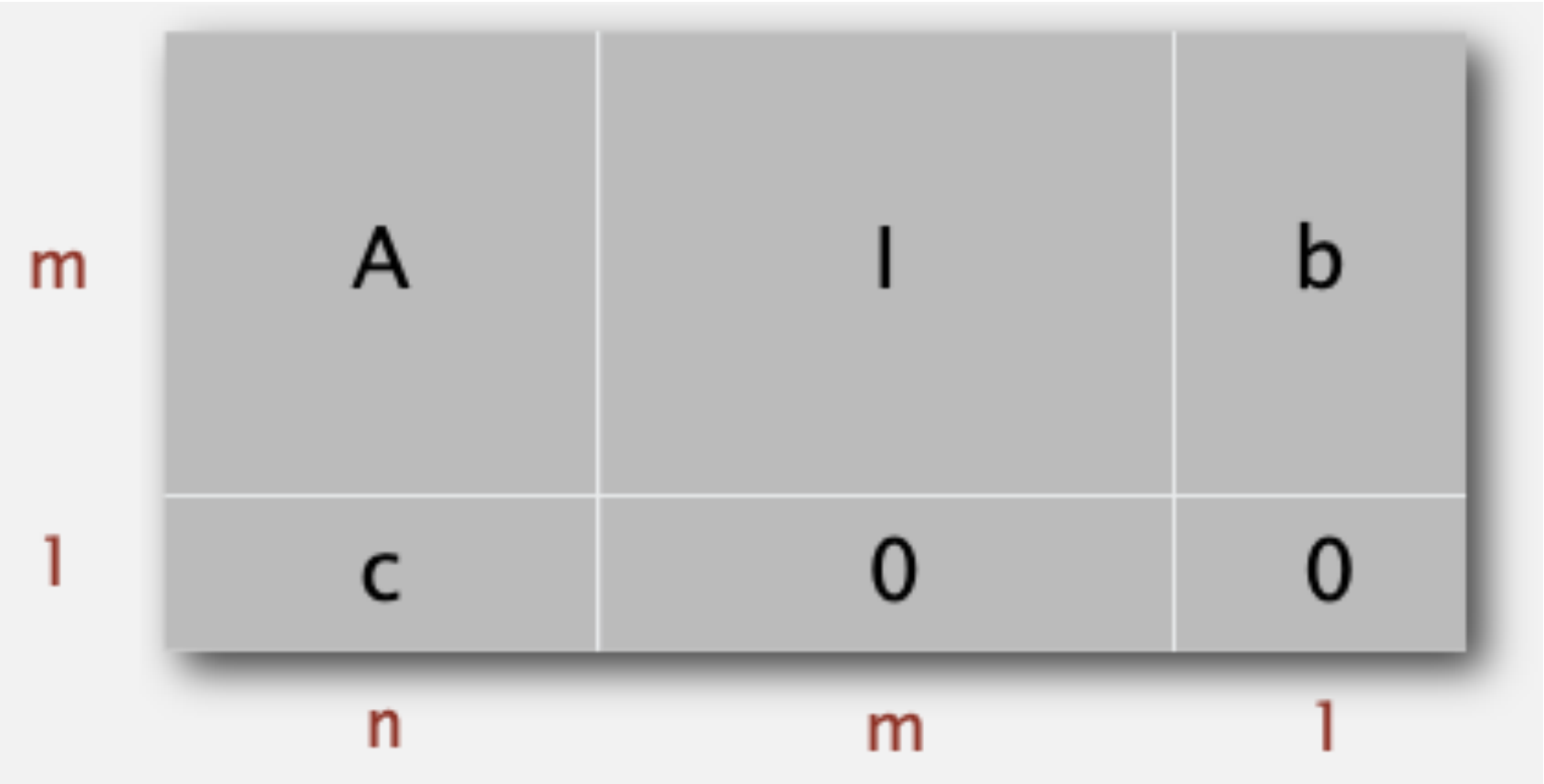
= 1190

≥ 0

m+1

5	15	1	0	0	480
4	4	0	1	0	160
35	20	0	0	1	1190
13	23	0	0	0	0

n+m+1



Initialization of 2D Array

```
public class Simplex {  
  
    private double[][] a;    // tableaux  
    private int m;          // number of constraints  
    private int n;          // number of original variables  
  
    public Simplex(double[][] A, double[] b, double[] c) {  
  
        m = b.length;  
        n = c.length;  
  
        for (int i = 0; i < m; i++)  
            if (!(b[i] >= 0)) throw new IllegalArgumentException("RHS must be nonnegative");  
  
        a = new double[m+1][n+m+1];  
        for (int i = 0; i < m; i++)  
            for (int j = 0; j < n; j++)  
                a[i][j] = A[i][j];  
        for (int i = 0; i < m; i++)  
            a[i][n+i] = 1.0;  
        for (int j = 0; j < n; j++)  
            a[m][j] = c[j];  
        for (int i = 0; i < m; i++)  
            a[i][m+n] = b[i];  
  
    }  
}
```

Input = Canonical Form:

max cx
st: $Ax \leq b$,
 $x \geq 0$
With $b \geq 0$

The slack variables that will
be our initial BFS

m	A	I	b
1	c	0	0
	n	m	1

Bland Rule: What variable to enter the basis for next pivot ?

Find entering column q using **Bland's rule**: index of first column whose objective function coefficient is positive (maximization problem).

// lowest index of a non-basic column with a positive cost

```
private int bland() {  
    for (int j = 0; j < m+n; j++)  
        if (a[m][j] > 0) return j;  
    return -1; // optimal  
}
```

	0	q	m+n
0			
p		+	
m		+	

Min Ratio rule: What variable should exist the BFS

// q is the variable (column) entering, we must discover p, the one going out
// find row p using min ratio rule (-1 if no such row)
// (smallest such index if there is a tie)

```
private int minRatioRule(int q) {  
    int p = -1; // leaving row  
    for (int i = 0; i < m; i++) {  
        if (a[i][q] <= 0) continue; // only positive entries  
        else if (p == -1) p = i;  
        else if ((a[i][m+n] / a[i][q]) < (a[p][m+n] / a[p][q]))  
            p = i; // row p has min ratio so far  
    }  
    return p;  
}
```

Pivoting on row p , column q

```
private void pivot(int p, int q) {  
    // p = variable exiting basis  
    // q = variable entering the basis  
  
    // everything but row p and column q  
    for (int i = 0; i <= m; i++)  
        for (int j = 0; j <= m+n; j++)  
            if (i != p && j != q) a[i][j] -= a[p][j] * (a[i][q] / a[p][q]);  
  
    // zero out column q  
    for (int i = 0; i <= m; i++)  
        if (i != p) a[i][q] = 0.0;  
  
    // scale row p  
    for (int j = 0; j <= m+n; j++)  
        if (j != q) a[p][j] /= a[p][q];  
    a[p][q] = 1.0;  
}
```



Main loop of the simplex

```
private void solve() {  
    while (true) {  
  
        // find entering column q  
        int q = bland();  
        if (q == -1) break;    // optimal  
  
        // find leaving row p  
        int p = minRatioRule(q);  
        if (p == -1) throw new ArithmeticException("Linear program is unbounded");  
  
        // pivot  
        pivot(p, q);  
    }  
}
```

But we were very lucky here 🙄

- Because it was very easy to find a first BFS

maximize	Z								
	13A	+	23B					- Z =	0
subject	5A	+	15B	+	S _C			=	480
to the	4A	+	4B			+	S _H	=	160
constraints	35A	+	20B				+	S _M	= 1190
	A	,	B	,	S _C	,	S _H	,	S _M ≥ 0

If any of those coefficient was negative, we wouldn't have been able to start with this BFS, because it would have violated the non-negativity constraints

Two Phase-Simplex

- Phase 1: find a BFS (using pivoting, with modified objective)
- Phase 2: optimize original objective starting with BFS of phase 1

Finding an initial BFS

$$\begin{array}{ll}\text{maximize} & 2x_1 - x_2 \\ \text{subject to} & 2x_1 - x_2 \leq 2 \\ & x_1 - 5x_2 \leq -4 \\ & x_1, x_2 \geq 0\end{array}$$

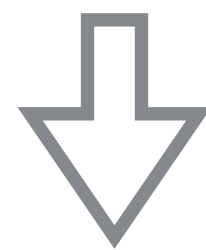
Standard form is (unfortunately not a BFS):

$$\begin{array}{llllll} z & = & & 2x_1 & - & x_2 \\ x_3 & = & 2 & - & 2x_1 & + & x_2 \\ x_4 & = & -4 & - & x_1 & + & 5x_2 \\ & & & x_1, x_2, x_3, x_4 & \geq & 0 \end{array}$$

Finding an initial BFS

- Put all the variables to the right such that you have constraints of type
 - $[0 = b_i + \dots \text{variables} \dots]$ with $b_i \geq 0$

$$\begin{array}{rclclcl} z & = & & 2x_1 & - & x_2 \\ x_3 & = & 2 & - & 2x_1 & + & x_2 \\ x_4 & = & -4 & - & x_1 & + & 5x_2 \\ & & & & x_1, x_2, x_3, x_4 & \geq & 0 \end{array}$$

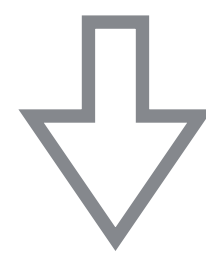


$$\begin{array}{rclclclcl} z & = & & 2x_1 & - & x_2 & & \\ 0 & = & 2 & - & 2x_1 & + & x_2 & - & x_3 \\ 0 & = & 4 & + & x_1 & - & 5x_2 & & + & x_4 \\ & & & & x_1, x_2, x_3, x_4 & \geq & 0 \end{array}$$

Finding an initial BFS

$$\begin{aligned} z &= 2x_1 - x_2 \\ 0 &= 2 - 2x_1 + x_2 - x_3 \\ 0 &= 4 + x_1 - 5x_2 + x_4 \\ x_1, x_2, x_3, x_4 &\geq 0 \end{aligned}$$

- Replace 0 in each constraint with a new fresh variable and minimize their sum. By construction you have a BFS for this modified problem so you can use simplex Algo to start optimizing it.



- If you arrive at $z=0$ you have a BFS to the initial problem, otherwise initial problem is unfeasible

$$\begin{aligned} z &= -x_5 - x_6 \\ x_5 &= 2 - 2x_1 + x_2 - x_3 \\ x_6 &= 4 + x_1 - 5x_2 + x_4 \\ x_1, x_2, x_3, x_4, x_5, x_6 &\geq 0 \end{aligned}$$

Finding an initial BFS

$$\begin{aligned} z &= \\ x_5 &= 2 - 2x_1 + x_2 - x_3 \\ x_6 &= 4 + x_1 - 5x_2 + x_4 \end{aligned}$$

$x_1, x_2, x_3, x_4, x_5, x_6 \geq 0$

Diagram illustrating the initial BFS setup. The constraints are shown as equations. The right-hand side values (2 and 4) are circled in green. Arrows indicate the substitution of x_5 and x_6 into the objective function z to eliminate them from the equations.

Original Objective

Initialization Objective

z	$=$	-6	$+$	$1x_1$	$+$	$4x_2$	$+$	$1x_3$	$-$	x_4
x_5	$=$	2	$-$	$2x_1$	$+$	x_2	$-$	x_3		
x_6	$=$	4	$+$	x_1	$-$	$5x_2$			$+$	x_4

Summary: Simplex is a two step method

1. Find a BFS

- by solving a modified problem with Simplex for which it is easy to find a BFS

2. Optimize starting from the BFS

Two phase simplex

```
public class TwoPhaseSimplex {

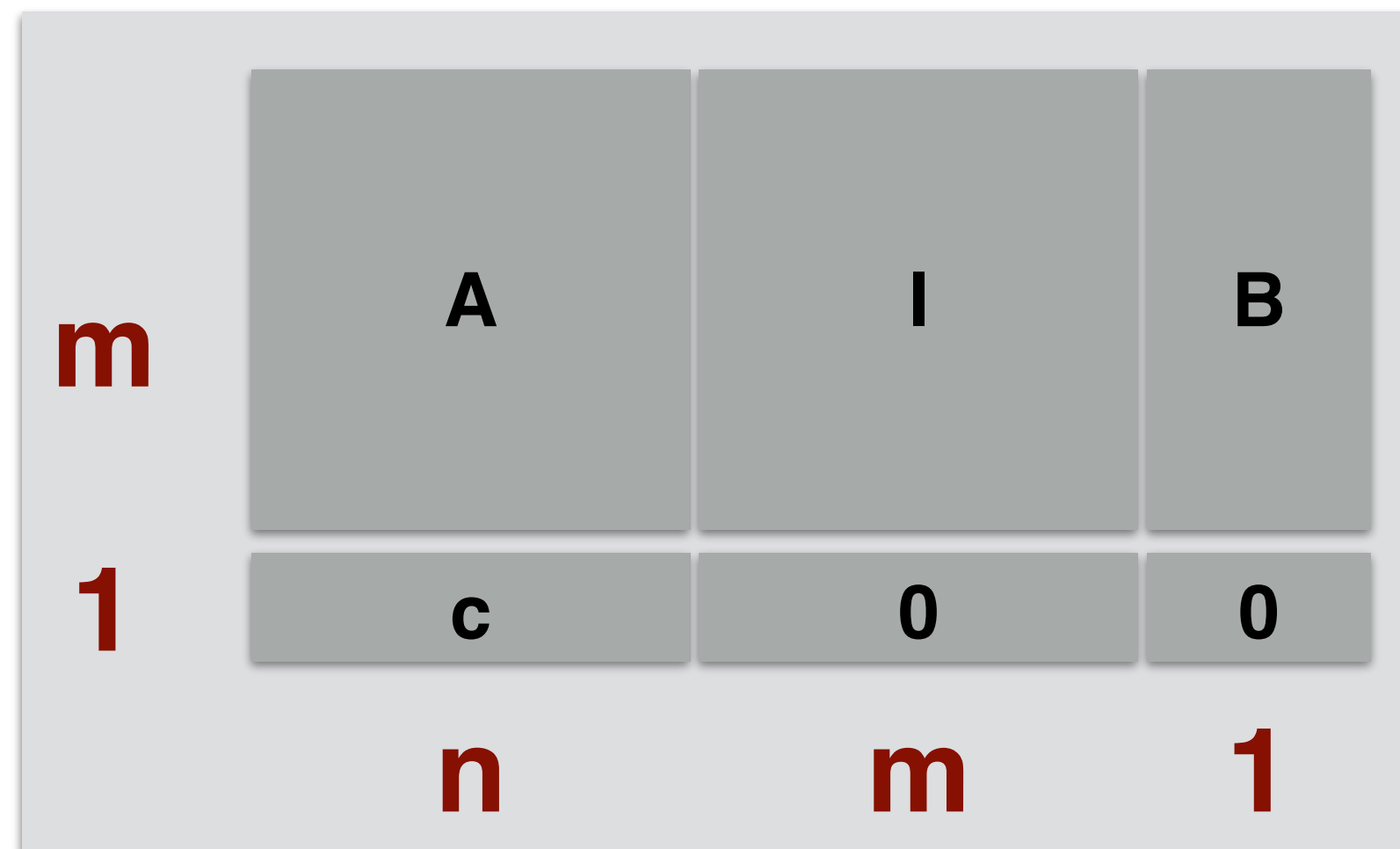
    private double[][] a;    // tableaux
    private int m;           // number of constraints
    private int n;           // number of original variables

    public TwoPhaseSimplex(double[][] A, double[] b, double[] c) {
        m = b.length;
        n = c.length;
        a = new double[m+2][n+m+m+1];

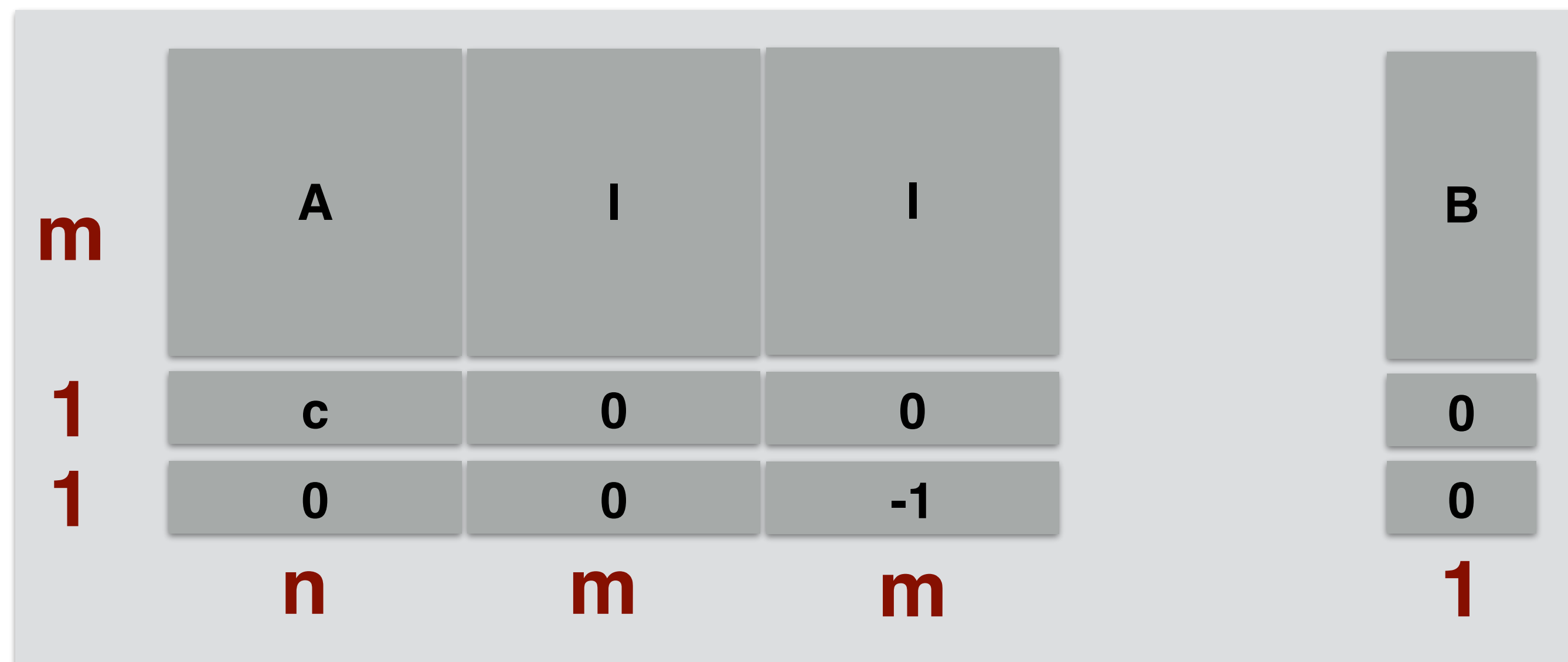
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = A[i][j];
        for (int i = 0; i < m; i++) a[i][n+i] = 1.0;
        for (int i = 0; i < m; i++) a[i][n+m+m] = b[i];
        for (int j = 0; j < n; j++) a[m][j] = c[j];
        // if negative RHS, multiply by -1
        for (int i = 0; i < m; i++) {
            if (b[i] < 0) {
                a[i][n+m+m] = -b[i];
                for (int j = 0; j <= n; j++)
                    a[i][j] = -a[i][j];
                a[i][n+i] = -1.0;
            }
        }
        // artificial variables form initial basis
        for (int i = 0; i < m; i++)
            a[i][n+m+i] = 1.0;
        for (int i = 0; i < m; i++)
            a[m+1][n+m+i] = -1.0;
        for (int i = 0; i < m; i++)
            pivot(i, n+m+i);

        basis = new int[m];
        for (int i = 0; i < m; i++)
            basis[i] = n + m + i;
    }
}
```

Original Simplex



Two Phase Simplex



The two phases

```
private void phase1() {
    while (true) {
        // find entering column q
        int q = bland1();
        if (q == -1) break; // optimal
        // find leaving row p
        int p = minRatioRule(q);
        assert p != -1 : "Entering column = " + q;

        // pivot
        pivot(p, q);
    }
    if (a[m+1][n+m+m] > 0)
        throw new ArithmeticException("Linear program is infeasible");
}
```

```
private void phase2() {
    while (true) {

        // find entering column q
        int q = bland2();
        if (q == -1) break; // optimal

        // find leaving row p
        int p = minRatioRule(q);
        if (p == -1)
            throw new ArithmeticException("Linear program is unbounded");

        // pivot
        pivot(p, q);
    }
}
```

// lowest index of a non-basic column with a positive cost - using artificial objective function

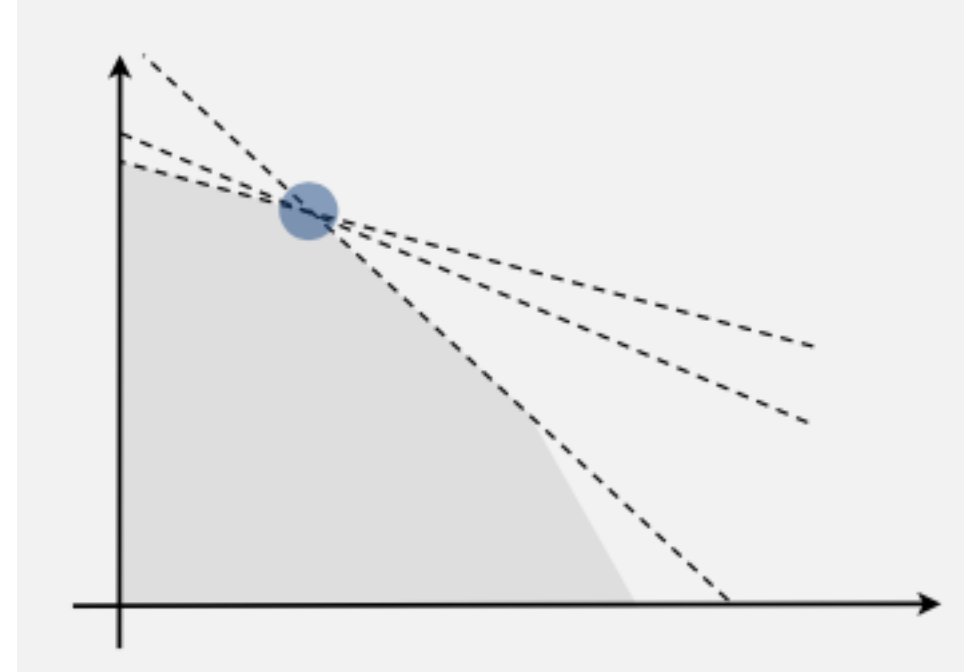
```
private int bland1() {
    for (int j = 0; j < n+m; j++)
        if (a[m+1][j] > EPSILON) return j;
    return -1; // optimal
}
```

// lowest index of a non-basic column with a positive cost

```
private int bland2() {
    for (int j = 0; j < n+m; j++)
        if (a[m][j] > EPSILON) return j;
    return -1; // optimal
}
```

Important remarks on computation

- Degeneracy: new basis, same extreme point (stalling quite frequent, don't worry too much)



- Cycling: get stuck by cycling through different bases that all correspond to same extreme point. If you use Bland rule, you are guaranteed to terminate 🎉

Simplex Time Complexity (wikipedia)

The simplex method is remarkably efficient in practice and was a great improvement over earlier methods such as [Fourier–Motzkin elimination](#). However, in 1972, Klee and Minty gave an example showing that the **worst-case complexity of simplex method as formulated by Dantzig is exponential time**. Since then, for almost every variation on the method, it has been shown that there is a family of linear programs for which it performs badly. It is an open question if there is a variation with [polynomial time](#), or even sub-exponential worst-case complexity.

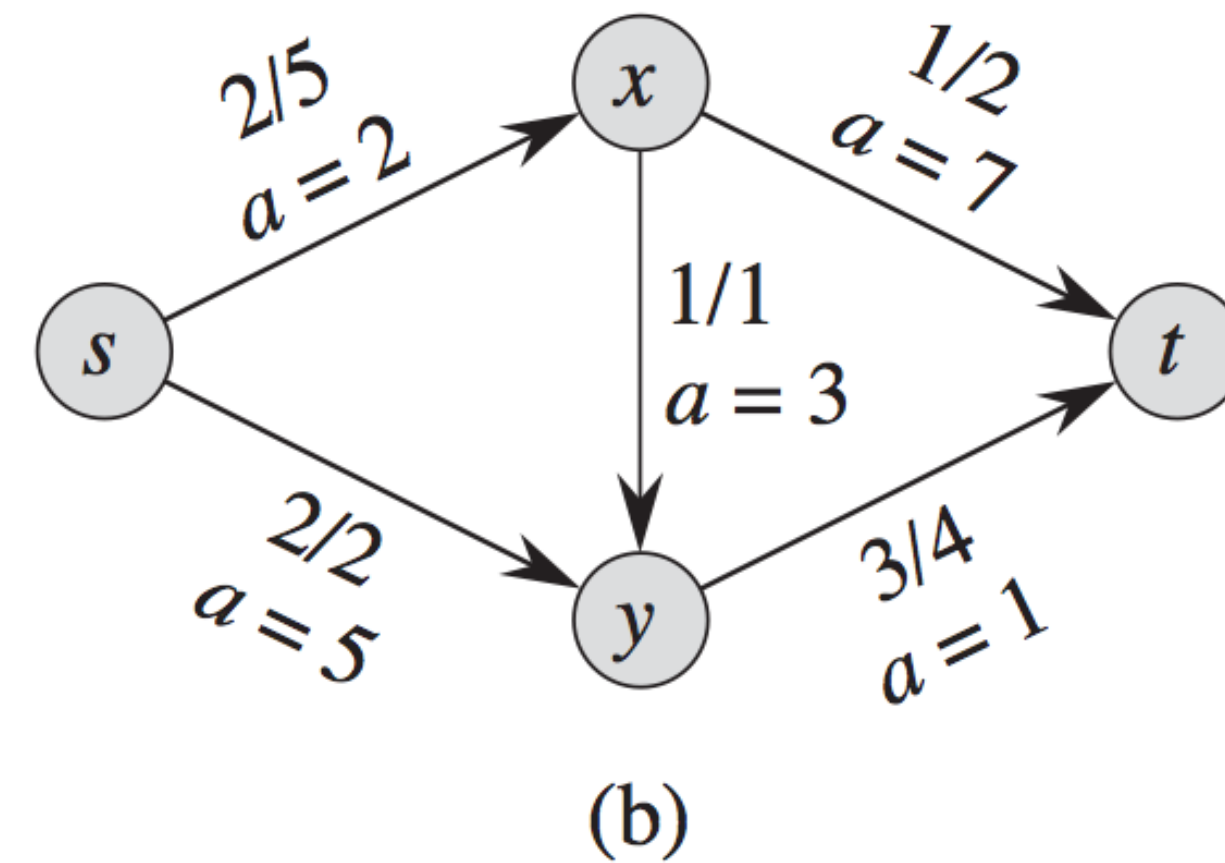
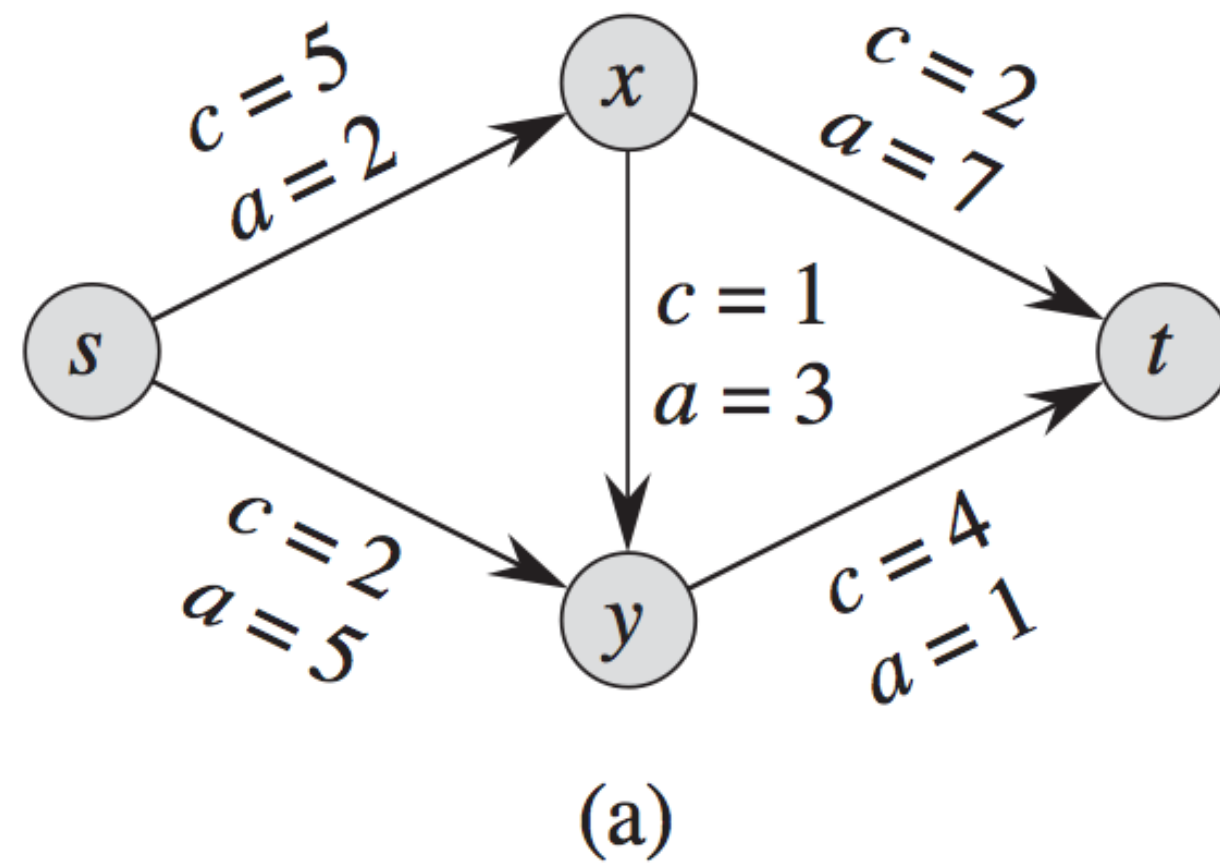
The simplex algorithm has **polynomial-time average-case complexity** under various [probability distributions](#), with the precise average-case performance of the simplex algorithm depending on the choice of a probability distribution for the [random matrices](#).

But LP solving is not NP hard, polynomial algorithms exist (Ellipsoid, Interior points). Those polynomial time algorithms are not necessarily better than simplex (and also less incremental).

Thus anytime you can reduce your problem to an LP, you know it can be solved in polynomial time. Example: Network Flow Problems

Example of problem well solved by LP

- Minimum Cost Flow
 - accommodate d unit of flow from s to t at minimum cost without exceeding the capacity.

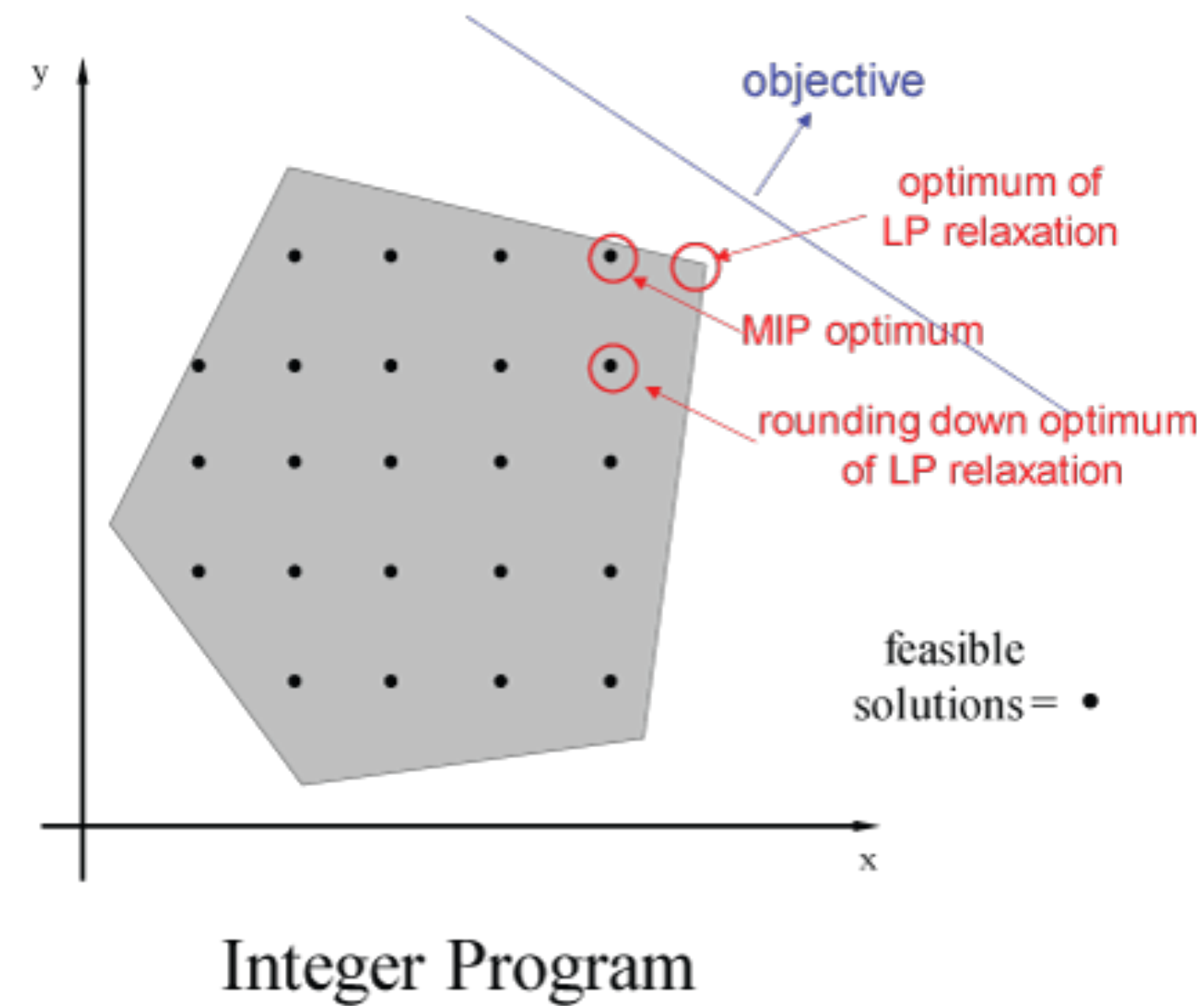


Integer Linear Programming (NP-Hard)

$$\begin{array}{ll}\text{maximize} & \sum_{j=1}^n c_j x_j \\ \text{subject to} & \sum_{j=1}^n a_{ij} x_j \leq b_i \text{ for } i = 1, 2, \dots, m\end{array}$$

$$x_j \in \mathbb{N} \text{ for } i = 1, 2, \dots, n$$

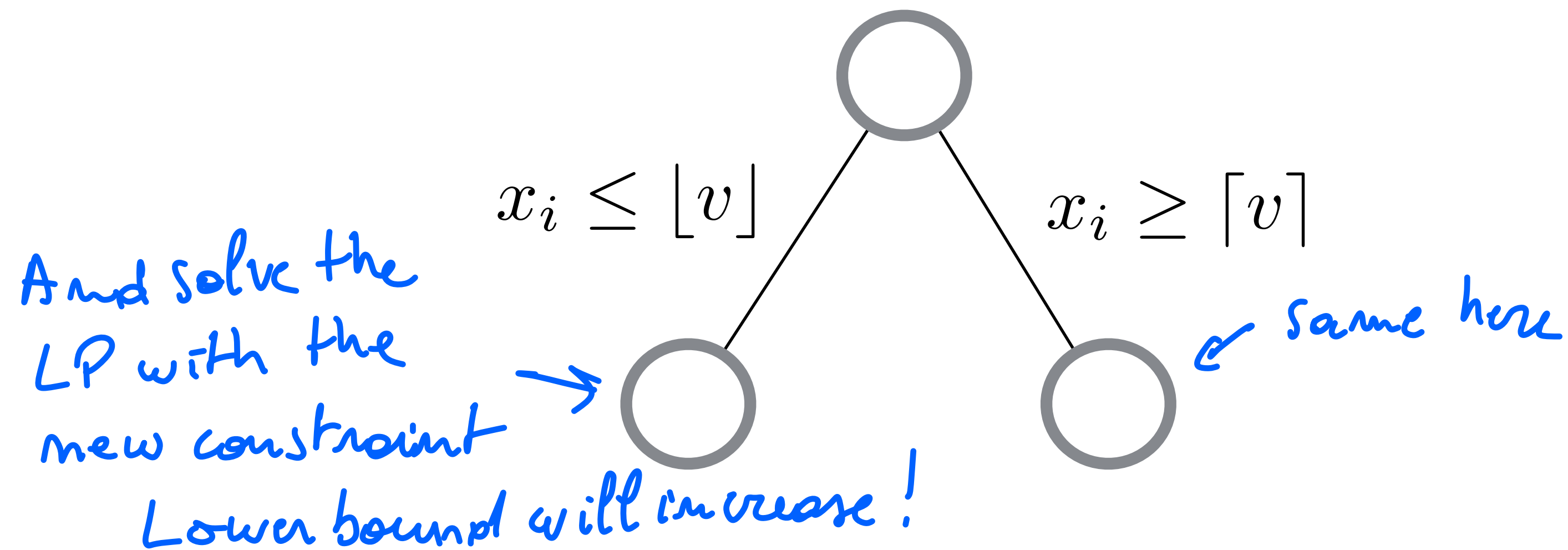
~~$\mathbb{R}$~~



How to solve using Branch & Bound
What do you suggest as relaxation/upper-bound?

Branch and Bound with LP relaxation

- If at the optimal solution of the linear programming relaxation, one variable is not an integer $x_i = v$
- Create two branches



- Adding those constraints can only decrease the upper bound (pruning of upper bound < best-so-far feasible solution)

ILP Branch and Bound DFS

```
function ILP_Solver(A, b, c):
```

```
    return BranchAndBoundDFS(A, b, c, -inf, [])
```

```
function BranchAndBoundDFS(A, b, c, bestValue, bestSolution):
```

```
    // Solve the LP relaxation
```

```
    solution, value = Solve_Simplex(A, b, c)
```

```
    // If no feasible solution, return
```

```
    if solution == null:
```

```
        return bestValue, bestSolution
```

```
    // If solution is integer, update bestValue and bestSolution if necessary
```

```
    if IsInteger(solution):
```

```
        if value > bestValue:
```

```
            bestValue = value
```

```
            bestSolution = solution
```

```
        return bestValue, bestSolution
```

```
    // Otherwise, branch on a non-integer variable
```

```
    variableToBranch = FindNonIntegerVariable(solution)
```

```
    floorValue = floor(solution[variableToBranch])
```

```
    ceilValue = ceil(solution[variableToBranch])
```

```
    // Add constraints to fix the variable at its floor value and solve the resulting problem
```

```
    A1, b1 = AddConstraint(A, b, variableToBranch, "<=", floorValue)
```

```
    bestValue, bestSolution = BranchAndBoundDFS(A1, b1, c, bestValue, bestSolution)
```

```
    // Add constraints to fix the variable at its ceiling value and solve the resulting problem
```

```
    A2, b2 = AddConstraint(A, b, variableToBranch, ">=", ceilValue)
```

```
    bestValue, bestSolution = BranchAndBoundDFS(A2, b2, c, bestValue, bestSolution)
```

```
    return bestValue, bestSolution
```

Simplex Inventor



George Dantzig

November 8, 1914 – May 13, 2005

Dantzig's roles in the discovery of LP and the simplex method are intimately linked with the historical circumstances, notably the Cold War and the early days of the Computer Age.

Richard Cottle, Ellis Johnson, and Roger Wets

Exercise

- Optimize this current LP to optimality
- How do you know it is optimal?

$$\begin{array}{rclclclcl} z & = & 20 & + & 2x_1 & - & x_2 & - & x_3 \\ x_4 & = & 25 & + & x_1 & - & x_2 & + & 3x_3 \\ x_5 & = & 12 & - & 2x_1 & - & 3x_2 & - & 4x_3 \\ x_6 & = & 15 & - & 3x_1 & + & x_2 & - & 4x_3 \\ & & & & x_1, x_2, x_3, x_4, x_5, x_6 & \geq & 0 \end{array}$$

Exercise

- Transform this into slack form and find (and) explain how to find an initial BFS.

$$\begin{aligned} \max \quad & 4x_1 + x_2 - x_3 \\ & x_1 + 3x_3 \leq 6 \\ & 3x_1 + x_2 + 3x_3 \leq 9 \\ & x_1 \geq 0, x_2 \geq 0, x_3 \geq 0. \end{aligned}$$

Best software for MIP and LP are commercial

- In practice, people rarely implement the simplex themselves because to make it efficient and robust you need to take advantage of scarcity, you need to deal with mathematical error and stability (artful engineering)
- Best tools are commercial softwares, free for Universities and students

The logo for FICO, consisting of the word "FICO" in a bold, blue, sans-serif font, with a small "TM" trademark symbol to the right.The logo for IBM CPLEX, featuring the word "IBM" in its characteristic blue horizontal stripes above the word "CPLEX" in a bold, red, sans-serif font, which is underlined.

GUROBI
OPTIMIZATION

Example Gurobi Model in Python

```
# Create optimization model
m = Model('netflow')

# Create variables
flow = m.addVars(commodities, arcs, obj=cost, name="flow")

# Arc capacity constraints
m.addConstrs(
    (flow.sum('*', i, j) <= capacity[i, j] for i, j in arcs), "cap")

# Flow conservation constraints
m.addConstrs(
    (flow.sum(h, '*', j) + inflow[h, j] == flow.sum(h, j, '*')
     for h in commodities for j in nodes), « node")

# Compute optimal solution
m.optimize()

# Print solution
if m.status == GRB.Status.OPTIMAL:
    solution = m.getAttr('x', flow)
```

The project

- Implement a new form of initialization, called big-M in case of negative coefficient and compare it with two-phase simplex

$$\begin{array}{ll}\max & cx - M(\mathbf{1}^\top x_a) \\ \text{subject to} & Ax + s + x_a = b \\ & s \geq 0 \\ & x \geq 0 \\ & x_a \geq 0\end{array}$$

- Model the network flow problem with LP and compare it with a dedicated algorithm (next week)